KAVERI FREIGHT & FULFILMENT PVT. LTD. · PUNE

CCAR-P · DOMAIN 3 · INTEGRATION · 19% — THE HEAVIEST DOMAIN · ~12 OF 63 QUESTIONS

The Loading Dock

Drill · eight code exhibits · layman glossary · Architecture Atlas · Dock Book (45 q) · sealed key · Memory Map

Days 5–8 of the 35-day campaign · Sat 3 – Tue 6 Oct 2026 · 14 study hours (5+5+2+2)

Eight objectives — and **four of them are new territory**: nothing on your ladder so far covers auth gaps, observability at scale, RAG design, or retrieval strategy. That is why this domain got the long weekend, the biggest question book, and its own glossary. Expect **VG** (vocabulary gap) to be your dominant miss family here; the glossary exists to starve it.

3.1	Tool/agent configuration — capability bloat	SOFT
3.2	AuthN / AuthZ — identify security gaps	NEW
3.3	Accuracy–latency trade-offs, justified	SOFT
3.4	Observability and monitoring at scale	NEW
3.5	RAG pipeline: chunking and indexing	NEW
3.6	Retrieval strategy by data shape and query	NEW
3.7	Integration mechanism: MCP / API / agent-to-agent	Carried over
3.8	Progressive discovery vs monolithic context	SOFT

THE FIVE LAWS OF DOMAIN 3 · EACH GETS ITS OWN BOX INSIDE; ALL FIVE LIVE ON THE MEMORY MAP

1. **Every tool you add is a door; know who holds the key.**
2. **Chunk along meaning, not length.**
3. **Retrieval follows the shape of the data, not the fashion.**
4. **You cannot fix what you cannot see.**
5. **Send the map first, the territory on demand.**

D1's laws remain in force — cited as “D1 Law 3” where a loop or budget point recurs. The Control Tower's shape (router → chains → agent tier) is settled canon; this book is about what it plugs into.

Holmes & Watson, back at the dock. — “It worked in the demo,” Watson, are the four most expensive words in logistics.

DAY A · Sat 3 Oct · 5 h — DRILL DAY I

- **30'** Miss ledger from D1 — read the Domain 3-adjacent rows out loud (tool design, MCP, context).
- **150'** Chapters 0–4: the dock brief, the standard connector, the bloat audit, the gate pass, chunking. Read E1–E4 against their decodes (code is read, not typed — then a stem is answered on it).
- **60'** Decode E3 and E4 line by line — auth and chunking are the week's two expensive lessons.
- **60'** First glossary pass: read every entry once; star the words you could not define cold.

TONIGHT YOU WILL

Tonight you will: explain to an empty chair why the fixed-size chunker gave Padma the wrong paragraph, and name the three server primitives without looking.

DAY B · Sun 4 Oct · 5 h — DRILL DAY II

- **30'** Miss ledger review. Same ritual.
- **150'** Chapters 5–9: hybrid retrieval, retrieval by shape, the dials, the dock cameras, the catalogue at the gate. Read E5–E8 against their decodes; one sabotage-in-your-head each.
- **60'** Sabotage labs on E5 (kill the BM25 half) and E7 (drop the trace id).
- **60'** Second glossary pass: cover the definitions, recite from the dock analogies; un-star what you now own.

TONIGHT YOU WILL

Tonight you will: say the contextual-retrieval numbers (35 → 49 → 67) and what each stage bought, and recite the five D3 laws.

DAY C · Mon 5 Oct · 2 h — BENCH DAY

- **15'** Miss ledger review.
- **90'** Dock Book: all 45 questions, one sitting, key sealed. Underline the tell before the options.
- **15'** Ledger every miss: objective, family, the tell you skipped.

TONIGHT YOU WILL

Tonight you will: know your D3 score by objective and whether VG really was the dominant family.

DAY D · Tue 6 Oct · 2 h — ATLAS DAY

- **15'** Miss ledger review — Day C rows included.
- **45'** Architecture Atlas A1–A7. Redraw A3 (retrieval-by-shape matrix) from memory.
- **45'** Retake every Dock Book miss cold; remaining starred glossary words get flashcard treatment.
- **15'** Finalise the Memory Map. Tomorrow: the Dispatcher's Prompt Book (D2).

TONIGHT YOU WILL

Tonight you will: have a clean D3 ledger, a redrawn A3, and zero starred glossary words.

CHAPTER 0

The Dock Brief — the tower is approved; now it must be connected

☂ THE BEAUTIFUL CRANE WITH NO POWER CABLE

Friday, the board approved a magnificent crane. Monday morning it stands on the dock, gleaming — connected to nothing. No power, no manifest feed, no gate passes for the operators, no cameras watching it work. A crane that lifts nothing is a sculpture. **Integration is the unglamorous week that turns architecture into operations** — and the exam weights it heaviest (19%) for exactly that reason: this is where real systems die.

Arjun's connection list for the week, which is also this book's table of contents:

#	Job at the dock	Objective	Chapter
1	Plug Claude into the TMS, the WMS and three carrier APIs — with a standard connector, not five custom forklifts	3.7	1
2	Audit the toolbag: nine tools proposed, most never used — every schema rides in the context window on every call	3.1	2
3	Give every door its own key. During the audit a carrier-portal token turns out to read EVERY customer's consignments — the Refund Heist's over-permissive ghost, back in Professional clothes	3.2	3
4	Make 400 SOP documents answerable — updated weekly, heading-structured, Hindi-English mixed. Shelve them (chunk), file them (index), find them (retrieve)	3.5 · 3.6	4–6
5	Set the dials: which model, how many chunks, what timeout — justified, not vibed	3.3	7
6	Watch 40,000 shipments a day: logs, traces, metrics — you cannot fix what you cannot see	3.4	8
7	Decide what rides in every prompt versus what is fetched on demand	3.8	9

The running thread: **KF-2481** is still in its customs hold. The paragraph that frees it — *SOP 214 §4, e-way bill re-filing* — exists. Chapter 4 watches a lazy chunking strategy retrieve the **wrong** paragraph first, with Padma on the phone; chapter 5 fixes it. The week ends with the bill filed and a lesson priced in rupees.

⚠ TRAP · "IT WORKED IN THE DEMO"

The demo had one user, ten documents, one API key taped over every door, and a developer watching the console. The dock has 40 dispatchers, 400 documents, three carriers and nobody watching. Every D3 stem is some version of the distance between those two sentences. When an option's defence is that it worked at small scale, look for what breaks at 40,000 a day.

CHAPTER 1 · OBJECTIVE 3.7

The standard connector — MCP, and when not to use it

☂ ONE DOCK CONNECTOR, NOT FIVE WELDED FORKLIFTS

Before shipping containers were standardised, every port needed custom cranes for every ship. The container did not make cargo better — it made **every pairing cheap**. MCP is the container standard for tools and data: one protocol, and any client can dock with any server. But standards have a price too: for a single fixed pairing you control end to end, a direct weld (plain API call) is sometimes less machinery. 3.7 grades knowing which.

An MCP server exposes three primitives — learn them as dock furniture: **tools** (executable actions — the forklifts), **resources** (context data — the manifests on the rack), **prompts** (reusable interaction templates — the laminated procedure cards). Clients discover each with */list, then fetch or call. Naming carries over from CCAR-F unchanged: mcp__server__tool.

EXHIBIT E1 · KAVERI_MCP.PY GROWN UP — TOOLS + A RESOURCE + A PROMPT

e1_kaveri_mcp.py

```python
from fastmcp import FastMCP
import json, pathlib

mcp = FastMCP("kaveri")                          # server name -> mcp__kaveri__*

@mcp.tool()
def get_shipment_status(consignment_id: str) -> dict:
    """Live status for one Kaveri consignment id."""
    return {"id": consignment_id, "status": "CUSTOMS_HOLD",
            "missing": "e-way bill"}

@mcp.tool()
def find_alternate_carriers(lane: str) -> list:
    """Carriers serving a lane, best first."""
    return ["BlueDart", "Delhivery"]

@mcp.resource("sop://214")                       # data, not action
def sop_214() -> str:
    """SOP 214 - customs holds and e-way bill re-filing."""
    return pathlib.Path("sops/sop_214.md").read_text(encoding="utf-8")

@mcp.prompt()
def customs_triage(consignment_id: str) -> str:
    """Laminated card: how to work a customs-hold ticket."""
    return (f"Work customs hold for {consignment_id}: check status, "
            "read sop://214, list options, never promise a date.")

if __name__ == "__main__":                       # the import-probe guard
    mcp.run()                                    # stdio transport by default
```

✎ PREDICT BEFORE RUN

Free probe first: `python -c "import e1_kaveri_mcp"` — predict what prints and why the server does NOT start. Then predict which of the three primitives a client would use to answer “what does SOP 214 say?” without spending a tool call.

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
python -c "import e1_kaveri_mcp"      # silence: the guard held
python e1_kaveri_mcp.py                # server runs on stdio, waits
# from a client: tools/list -> 2 tools · resources/list -> sop://214
# prompts/list -> customs_triage
```

DECODE · LINE BY LINE, ONE STORY

1–4	The story: **the dock gets its standard connector.** One import, one server object, and the name “kaveri” becomes the namespace — every tool will surface to clients as mcp__kaveri__..., exactly the naming CCAR-F drilled.
6–15	Two forklifts. The decorator turns an ordinary function into a discoverable tool: name from the function, schema from the type hints, description from the docstring. **The docstring IS the interface** — it is what the model reads when deciding to call. A lazy docstring here is a mis-dialled phone call later.
17–20	The manifest rack. A **resource** is data a client can read into context — no action, no side effects. SOP 214 rides here because reading a document should not cost a tool-call round-trip. Tools act; resources inform. Mixing them up is a 3.7 distractor shape.
22–26	The laminated card. A **prompt** is a reusable template the client can fetch and apply — the triage procedure written once, versioned on the server, instead of pasted into forty dispatchers' heads.
28–29	The guard from CCAR-F, still earning: import the file and mcp.run() never fires — the free import-probe stays free. Run it as a script and the server speaks stdio: a **local** server, one client, no network. The remote flavour is Streamable HTTP — chapter 3 puts authentication on it.

⚒ SABOTAGE LAB · BREAK EXACTLY ONE VARIABLE

Break line 7: change the type hint to consignment_id: int. Predict: does the server crash, or does the generated schema change — and what does the model now send? Probe with tools/list, compare the inputSchema before and after, fix it back. Lesson: **in FastMCP the type hints write the contract; a wrong hint is a wrong phone book.**

⇄ DRIFT · THE MCP PROTOCOL MOVED UNDER THE WORKBOOKS (SPEC 2026-07-28)

Workbook / common teaching: Common teaching: client and server perform an initialize handshake and stay stateful; **sampling** and **logging** are core client primitives; remote servers use the SSE transport.
Current docs (checked Sep 2026): Current spec (2026-07-28): the protocol is **stateless** — every request carries version and capabilities in _meta, with an optional server/discover request replacing the old handshake role; **sampling and logging are deprecated** (integrate the LLM directly; log to stderr or OpenTelemetry); change notifications are opt-in via subscriptions/listen; transports are **stdio** and **Streamable HTTP** — SSE-as-transport is gone. Client primitive still current: elicitation.

MCP, DIRECT API, OR AGENT-TO-AGENT — THE 3.7 DECISION

The stem says	Choose	Because
Many clients (KAVI, control tower, Arjun's IDE) must reach the same tools; tool list changes	MCP server	One connector, N clients; discovery keeps clients current
One fixed integration, one owner, tight latency budget, no reuse foreseen	Direct API/CLI call in code	A standard buys nothing here; a weld is less machinery
The other side is itself an agent with judgement (a carrier's booking agent negotiating slots)	Agent-to-agent	You are exchanging goals and proposals, not calling functions
A carrier offers only a REST API and you control the agent side	Wrap the API as an MCP tool	The weld hides behind the standard connector once, then N clients benefit

⚠ TRAP · STANDARD FOR STANDARD'S SAKE

"Migrate all five integrations to MCP for consistency" is OE when three of the five have exactly one client and will never have another. The container revolution paid off at the pairing count, not at the container count. One client, one server, no churn → the weld was fine.

CHAPTER 2 · OBJECTIVE 3.1

The bloat audit — every schema rides in every prompt

☂ THE TOOLBELT THAT DRAGS ON THE FLOOR

A new fitter proudly wears every tool the store sold him. By noon his belt drags; by day two he reaches for the wrong wrench because six look alike. The old fitter carries three tools and finishes first. The belt is the context window: **every tool's name, description and schema is paid for on every call, used or not** — and lookalike tools make the model fumble.

Capability bloat is CCAR-F tool design meeting Professional economics. Two costs, both graded: the **token bill** (schemas ride along on every request) and the **confusion bill** (overlapping descriptions raise wrong-tool rates). The audit is code, not opinion:

EXHIBIT E2 · MEASURE THE BELT, THEN TRIM IT

e2_bloat_audit.py

```python
import json, anthropic
client = anthropic.Anthropic()

def schema_tokens(tools):                       # price the belt itself
    return client.messages.count_tokens(        # count_tokens is FREE
        model="claude-sonnet-5",
        tools=tools,
        messages=[{"role": "user", "content": "x"}]).input_tokens

ALL9 = json.load(open("tools_all9.json"))       # the proud new belt
CORE3 = [t for t in ALL9 if t["name"] in
         {"get_shipment_status", "find_alternate_carriers",
          "read_sop"}]                          # the old fitter's belt

bill9, bill3 = schema_tokens(ALL9), schema_tokens(CORE3)
print(f"9 tools: {bill9} tokens / call")
print(f"3 tools: {bill3} tokens / call")
daily = 40_000
print(f"belt tax: {(bill9-bill3) * daily:,} tokens/day for tools "
      f"the logs say were called 0 times last week")
```

✎ PREDICT BEFORE RUN

Before running: guess the token cost of one tool schema (name + description + JSON schema). Then predict the daily belt tax at 40,000 calls. Most people guess low by 3×.

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
9 tools: 2,314 tokens / call
3 tools: 811 tokens / call
belt tax: 60,120,000 tokens/day for tools the logs say were called 0 times last week
```

DECODE · LINE BY LINE, ONE STORY

1–8	The story: **Arjun prices the belt before wearing it.** count_tokens is the free probe of the API world — it prices a request without running it. CCAR-F's "free probe before paid call" discipline, now doing capacity planning.
10–13	Two belts on the bench: the nine-tool proposal and the three tools last week's logs say were actually called. Note what decides membership: **the logs**, not anyone's feelings. Chapter 8's cameras are what make this audit possible at all — the laws interlock.
15–19	The arithmetic that wins meetings: ~1,500 tokens of unused schema × 40,000 calls = 60 million tokens a day of pure belt tax — before counting the wrong-wrench errors. Trim, and re-add a tool the day a log line asks for it.

The subtler bloat: **lookalikes**. `lookup_order` and `get_shipment_status` overlap enough that the model sometimes dials the wrong one. Merging or sharpening descriptions is a bloat fix too — the belt should have *distinct* tools, not just few.

LAW 1 · EVERY TOOL YOU ADD IS A DOOR; KNOW WHO HOLDS THE KEY

A tool is simultaneously a token cost (its schema, every call), a confusion cost (its lookalikes), and a security surface (chapter 3). The audit question is never "might it be useful?" — it is "who used it last week, and who can walk through it?"

⚠ TRAP · TRIMMING THE LOAD-BEARING TOOL

The reflex must not over-learn: when the stem's task genuinely needs a capability — escalation, say, used once a month but on the worst day of the month — usage counts alone don't retire it. Trim by logs AND consequence. escalate_to_human stays on the belt at zero weekly calls, because the day it is needed is the day that pays for the year.

CHAPTER 3 · OBJECTIVE 3.2

The gate pass — who you are is not which bays you may enter

☂ THE GATE PASS AND THE MANIFEST

At the dock gate, the guard checks two different papers. Your **gate pass** proves who you are — that is authentication (authN). The **manifest** says which bays your truck may enter — that is authorization (authZ). The heist crews of this world are rarely stopped by the first paper: they walk in with a real pass whose manifest someone forgot to narrow. The Refund Heist's ghost is exactly this: a valid credential with too many bays on it.

New territory, so slowly: **authN** = verifying identity (API keys, OAuth tokens on Streamable HTTP, service accounts for machine callers). **authZ** = what that identity may do (scopes, least privilege, per-tool checks). The exam's 3.2 stems almost always hide an authZ gap behind a working authN story — everything logs in fine, and that is the trap.

EXHIBIT E3 · THE SCOPED KEY, CHECKED IN CODE

e3_gate_pass.py

```
SCOPES = {                                  # the manifest, in code
    "kavi-agent":        {"status:read", "carriers:read"},
    "refund-desk":       {"status:read", "refund:write"},
    "carrier-portal-v1": {"status:read", "*"},   # <- the ghost: wildcard
}
REQUIRED = {                                      # every door names its key
    "get_shipment_status":    "status:read",
    "find_alternate_carriers":"carriers:read",
    "process_refund":         "refund:write",
}

def authorize(caller: str, tool: str) -> None:
    granted = SCOPES.get(caller, set())
    needed = REQUIRED[tool]                       # unknown tool -> KeyError: good
    if needed not in granted and "*" not in granted:
        raise PermissionError(
            f"{caller} lacks {needed} for {tool}")   # structured, loggable

def run_tool(caller, tool, args):
    authorize(caller, tool)                       # BEFORE the tool, always
    audit_log(caller=caller, tool=tool, args=args)   # chapter 8 will read this
    return TOOLS[tool](**args)

# the fix, after the audit:
SCOPES["carrier-portal-v1"] = {"status:read"}    # wildcard revoked
```

✎ PREDICT BEFORE RUN

Predict the three outcomes before running: kavi-agent calling process_refund; refund-desk calling process_refund; carrier-portal-v1 calling process_refund BEFORE line 25 runs. The third answer is the whole chapter.

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
kavi-agent -> process_refund      PermissionError: lacks refund:write
refund-desk -> process_refund     OK (scope matches)
carrier-portal-v1 -> process_refund   OK  <- silent, valid, catastrophic
# after line 25: PermissionError. The audit closed the bay.
```

DECODE · LINE BY LINE, ONE STORY

1–5	The manifest as a dict: each caller, its bays. Line 4 is **the ghost** — a wildcard scope issued eighteen months ago “to get the demo working”. It authenticates perfectly. Every 3.2 stem hides one of these; your job is reading credential TABLES, not login flows.
6–10	Every door names the one key that opens it. This table is the security model — small enough to review in a meeting, which is precisely the point. What cannot be tabled cannot be audited.
12–17	The check: set membership, in code, **before** the tool body. Note what it is not: a system-prompt instruction saying “only the refund desk may refund”. CCAR-F's law — config beats instruction — is now called authZ and graded at Professional weight. The error is structured and loggable, not a silent False.
19–22	Order of operations is the lesson: authorize → audit_log → tool. The log line writes even for denied calls — tomorrow's chapter 8 dashboard has a “denials by caller” panel, and a spike on it is how you catch a probe in progress.
24–25	The fix is one line because the model was scopes-in-a-table. The heist replay now dies at the gate with a named reason. Least privilege is not a slogan; it is a small diff.

⚒ SABOTAGE LAB · BREAK EXACTLY ONE VARIABLE

Break line 15: change the check to if needed not in granted (drop the wildcard clause). Predict which caller's behaviour changes and in which direction. Run all three probes. Lesson: **a security check you didn't test both ways is a guess** — and yes, this sabotage makes the system SAFER; notice that feeling and log it.

On the wire: local stdio servers inherit the launching user's identity — the gate is the OS. Remote Streamable HTTP servers authenticate with standard HTTP: bearer tokens, API keys, and **OAuth is MCP's recommended route** for obtaining them. AuthN choice is transport-shaped; authZ is yours either way.

⚠ TRAP · AUTH THEATER

Four shapes that sound diligent and secure nothing, in rising order of exam frequency: a system prompt forbidding an action the tool list still allows; a wildcard scope “to be revoked later”; authenticating the USER but running every tool as one god-mode service account; logging denials nobody reads. The tell in 3.2 stems is a credential that works — then ask what it works ON.

CHAPTER 4 · OBJECTIVE 3.5A

Shelving the SOPs — chunking is where retrieval is won or lost

☂ HOW YOU SHELVE THE BOXES

A warehouse that saws every incoming shipment into equal half-metre boxes has beautiful, uniform shelves — and a sofa in six boxes on four aisles. When the order for the sofa comes, three trips and a missing armrest. The rival warehouse shelves **along the seams**: one product, one box, one label. Chunking is shelving: **cut along the document's meaning** — its headings, its sections — **not along a tape measure**.

Monday, 11:40. Padma on the phone. Arjun's first RAG build cut all 400 SOPs into fixed 500-token chunks. The query “e-way bill re-filing” retrieved a chunk that *starts mid-sentence in §3 (bill cancellation)* and ends before §4's actual procedure. KAVI confidently read out the cancellation steps. Padma now has a cancelled e-way bill. This chapter is that mistake, dissected and priced.

EXHIBIT E4 · CHUNKING THREE WAYS ON SOP 214

e4_chunking.py

```python
import re
SOP = open("sops/sop_214.md", encoding="utf-8").read()

def fixed(text, size=500):                        # the tape measure
    words = text.split()
    return [" ".join(words[i:i+size]) for i in range(0, len(words), size)]

def by_heading(text):                             # the seams
    parts = re.split(r"(?m)^(#{1,3} .+)$", text) # keep the headings
    chunks, title = [], "SOP 214"
    for p in parts:
        if p.startswith("#"): title = p.strip("# ").strip()
        elif p.strip():
            chunks.append(f"[SOP 214 - {title}]\n{p.strip()}")
    return chunks

def with_overlap(chunks, tail=40):                # a photo of the neighbour
    out = []
    for i, c in enumerate(chunks):
        prev_tail = " ".join(chunks[i-1].split()[-tail:]) if i else ""
        out.append((prev_tail + "\n" + c).strip())
    return out

for name, ch in [("fixed", fixed(SOP)), ("heading", by_heading(SOP))]:
    hit = [c for c in ch if "re-filing" in c and "Step 1" in c]
    print(f"{name:8s}: {len(ch):3d} chunks | full procedure in one chunk: "
          f"{bool(hit)}")
```

✎ PREDICT BEFORE RUN

Predict both printed lines: chunk counts (SOP 214 is ~4,000 words with 9 headed sections) and, for each strategy, whether the complete re-filing procedure lands inside ONE chunk. Then predict which strategy retrieves the CANCELLATION section for the re-filing query, and why the two sections' vocabulary makes that confusion easy.

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
fixed   :   8 chunks | full procedure in one chunk: False
heading :   9 chunks | full procedure in one chunk: True
# the fixed cut lands mid-#3; "e-way bill" appears in both #3 and #4,
# so the retriever cannot tell cancellation from re-filing. The heading
# cut carries "[SOP 214 - S4 Re-filing]" INSIDE the chunk text.
```

DECODE · LINE BY LINE, ONE STORY

1–2	The story: **re-shelving the warehouse that hurt Padma.** One real document, three shelving policies, same retrieval question at the end.
4–6	The tape measure: every 500 words, cut. Simple, uniform, and meaning-blind — §3 and §4 end up sharing a box, which is exactly how a cancellation procedure answers a re-filing question. Fixed-size is not evil; it is the default that must be argued for, not from.
8–15	The seams. The regex keeps the headings, and line 14 does the quiet trick that carries the whole chapter: the section title is written **into the chunk text itself** — [SOP 214 – §4 Re-filing]. A chunk that knows where it came from embeds better, keyword-matches better, and cites itself. Hold this thought: chapter 5 reveals it as a baby step toward contextual retrieval.
17–22	The photo of the neighbour: each box carries the last 40 words of the previous one, so a sentence straddling a seam exists whole in at least one box. Overlap costs duplicate tokens — a dial, not a default.
24–27	The verdict, printed: same document, same question — one policy can answer it in one retrieval, one cannot. **Retrieval quality was decided at ingest time**, before any embedding model or clever reranker got a vote.

⚒ SABOTAGE LAB · BREAK EXACTLY ONE VARIABLE

Break line 9's regex: match only `^#` (top-level headings). Predict the new chunk count and whether the §4 chunk is now too big to be a precise answer. Lesson: **chunk granularity should match query granularity** — dispatchers ask section-level questions, so section-level boxes.

LAW 2 · CHUNK ALONG MEANING, NOT LENGTH

Cut on the document's own structure (headings, sections, tickets) and write each chunk's origin into its text. Size and overlap are dials tuned per corpus; the seam is the principle. A retrieval system's ceiling is set at the shelving stage — everything downstream can only lose height.

⚠ TRAP · RE-CHUNKING AS A CURE FOR EVERYTHING

Once burned, teams re-chunk at every retrieval miss. But when the miss pattern is “right section, wrong document” the gap is usually indexing or hybrid weighting (chapter 5), and when it is “right chunk retrieved, wrong answer written” the gap is prompt assembly, not shelving. Diagnose by where the right paragraph fell out of the pipeline, then fix that stage.

CHAPTER 5 · OBJECTIVE 3.5B / 3.6A

The ledger and the foreman — indexing and hybrid retrieval

☂ ASK THE OLD FOREMAN AND CHECK THE REGISTER

Two ways to find a box in the warehouse. The **register** lists exact words — ask for “e-way bill” and it returns every box whose label contains those words, and nothing else. The **old foreman** has walked these aisles thirty years; ask for “customs paperwork problem” and he points at the right aisle even though no label says those words — he files by *meaning*. The register is literal and misses synonyms; the foreman is fuzzy and sometimes points at the plausible-but-wrong aisle. **The trick the whole industry converged on: ask both, trust where they agree.**

Names for the furniture: the register is **keyword search (BM25)**; the foreman is **semantic search** over **embeddings** — vectors that place similar meanings near each other in space, stored in a **vector index**. Fusing both rankings is **hybrid search**; having a slower, smarter reader re-sort the shortlist is **reranking**.

EXHIBIT E5 · THE HYBRID, SMALL ENOUGH TO HAND-TYPE

e5_hybrid.py (norm() scales keyword scores to 0–1; CHUNKS from E4)

```
1   import math, os
2   from collections import Counter
3   DRY_RUN = os.environ.get("DRY_RUN") == "1"
4
5   def embed(text):                                # stub in DRY_RUN: hash-based
6       if DRY_RUN:
7           return [hash(w) % 97 / 97 for w in sorted(set(text.lower().split()))][:8]
8       return real_embed(text)                     # a paid embedding call
9
10  def cosine(a, b):
11      n = min(len(a), len(b))
12      dot = sum(x*y for x, y in zip(a[:n], b[:n]))
13      return dot / (math.sqrt(sum(x*x for x in a[:n]) or 1) *
14                    math.sqrt(sum(y*y for y in b[:n]) or 1))
15
16  def bm25ish(q, chunk):                          # the register, simplified
17      words = Counter(chunk.lower().split())
18      return sum(words[w] for w in q.lower().split())
19
20  def hybrid(query, chunks, k=3, w_sem=0.6):
21      qv = embed(query)
22      scored = []
23      for c in chunks:
24          s_sem = cosine(qv, embed(c))            # the foreman's opinion
25          s_kw = bm25ish(query, c)                # the register's count
26          scored.append((w_sem * s_sem + (1 - w_sem) * norm(s_kw), c))
27      return [c for _, c in sorted(scored, reverse=True)[:k]]
28
29  top = hybrid("procedure to re-file an e-way bill", CHUNKS)
30  print([c.splitlines()[0] for c in top])         # the shortlist's labels
```

✎ PREDICT BEFORE RUN

Two queries, predict each shortlist's top label BEFORE running: (1) “e-way bill re-filing” — which half of the hybrid does the work when the exact phrase exists in the corpus? (2) “customs paperwork stuck, what do I do” — no SOP contains the word 'stuck'; which half rescues it? Write both predictions, then run both.

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
q1: ['[SOP 214 - S4 Re-filing]', '[SOP 214 - S3 Cancellation]', ...]
    # exact phrase -> the register (keyword) half carried it
q2: ['[SOP 214 - S1 What a customs hold is]', '[SOP 214 - S4 Re-filing]', ...]
    # zero keyword overlap -> the foreman (semantic) half carried it
# kill either half (sabotage) and one of the two queries fails.
```

DECODE · LINE BY LINE, ONE STORY

1–8	The story: **the warehouse gets both a register and a foreman.** The embed stub keeps DRY_RUN free — a fake-but-deterministic vector per text, so the plumbing is testable before any paid embedding call. Same discipline, new domain.
10–14	Cosine similarity is the foreman's “how near is this aisle” — the angle between two meaning-vectors. You never compute it in the exam; you say what it measures: closeness of meaning, not sharing of words.
16–18	The register, honestly simplified: count query-word occurrences. Real BM25 also rewards rare words and damps long documents, but the exam-relevant property is present: **literal, exact, synonym-blind**.
20–27	The fusion. One weight, 0.6 semantic, blends the two rankings; top-k boxes come back. Each query type leans on a different half — which is the entire argument for hybrid: **each half fails alone, on different queries, silently**.
29–30	Printing labels, not full chunks — because E4 wrote origins into the text, the shortlist is self-describing. Small habits from the shelving chapter keep paying.

⚒ SABOTAGE LAB · BREAK EXACTLY ONE VARIABLE

Break line 26: set w_sem=1.0 (kill the register). Predict which of the two probe queries degrades and HOW — wrong section, or right section at lower rank? Run, ledger the result, restore. Lesson: **single-channel retrieval fails on a query shape, not at random** — which is why the failure hides in demos.

⇄ DRIFT · WHAT "GOOD RAG" MEANS HAS MOVED

Workbook / common teaching: Common teaching: chunk → embed → vector search → top-5 into the prompt; keyword search is the legacy thing RAG replaced.
Current docs (checked Sep 2026): Anthropic's current guidance (Contextual Retrieval): prepend chunk-situating context before BOTH embedding and BM25 — keyword search is a first-class half, not legacy. Measured on top-20 retrieval failure rate: contextual embeddings alone cut it **35%** (5.7→3.7%), plus contextual BM25 **49%** (→2.9%), plus reranking **67%** (→1.9%). The reranking shape: retrieve **150** candidates, rerank, pass **top-20** — and top-20 beat top-5 and top-10 outright. And the pre-question: under ~**200k** tokens of corpus (~500 pages), skip RAG — put it all in the prompt behind prompt caching.

E4's little habit — writing [SOP 214 – §4] into each chunk — is contextual retrieval's baby form: the full technique has a model write a situating sentence per chunk at ingest (once, cacheable), so both the foreman and the register see "this chunk is from the customs SOP, about re-filing" instead of a bare paragraph. Same principle, industrial grade.

CHAPTER 6 · OBJECTIVE 3.6B

Three questions, three routes — retrieval follows the data's shape

☂ YOU DON'T ASK THE FOREMAN FOR BAY NUMBERS

"Where is consignment KF-2481?" — you look it up in the tracking system; asking the wise old foreman to *reminisce* about it would be absurd. "How do I re-file an e-way bill?" — now you want the foreman and the register. "How many shipments ran late this week?" — neither: that is the accountant's job, a sum over rows. **Three question shapes, three completely different machines** — and the most expensive mistake in this domain is feeding a lookup or a sum into a similarity search.

EXHIBIT E6 · THE RETRIEVAL ROUTER

e6_shape_router.py

```python
import re

def route(query: str) -> str:
    if re.search(r"\bKF-\d{4,}\b", query):      # an ID -> exact world
        return "structured_lookup"
    if re.search(r"how many|count|average|trend|this week|per day",
                 query, re.I):                   # aggregate language
        return "sql_aggregate"
    return "hybrid_rag"                          # policy/procedure prose

HANDLERS = {
    "structured_lookup": lambda q: tms.get(extract_id(q)),   # no RAG!
    "sql_aggregate":     lambda q: db.run(to_sql(q)),        # count rows
    "hybrid_rag":        lambda q: answer_from(hybrid(q, CHUNKS)),
}

for q in ["Where is KF-2481 right now?",
          "How many shipments ran late this week?",
          "What is the procedure to re-file an e-way bill?"]:
    print(f"{route(q):18s} <- {q}")
```

✎ PREDICT BEFORE RUN

Predict all three printed routes. Then the costly counterfactual, in the notebook: what does hybrid_rag return for 'How many shipments ran late this week?' — and why is a fluent wrong number worse than an error?

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
structured_lookup  <- Where is KF-2481 right now?
sql_aggregate      <- How many shipments ran late this week?
hybrid_rag         <- What is the procedure to re-file an e-way bill?
# hybrid_rag on the count question retrieves chunks that MENTION lateness
# and the model composes a plausible number. Fluent, cited-looking, wrong.
```

DECODE · LINE BY LINE, ONE STORY

1–9	The story: **one desk, three question shapes, a router at the front.** This is D1's routing pattern (E3 there) reborn one level down the stack — cheap classification first, specialist machinery second. Patterns compose across domains; the exam loves noticing that.
11–15	Three handlers, three worlds. The comment on line 12 is the chapter's thesis: an ID lookup is **exact-answer** work — the tracking database holds THE answer, and similarity search can only return things LIKE the answer. Semantic machinery on exact questions doesn't merely waste money; it converts certainty into probability.
17–21	The three probes, one per shape. Say the mapping aloud, it is exam-recitable: ID or code → structured lookup · count/trend → aggregate query · policy/procedure prose → hybrid RAG · exact phrase or code snippet → keyword-weighted · vague symptom prose → semantic-weighted.

LAW 3 · RETRIEVAL FOLLOWS THE SHAPE OF THE DATA, NOT THE FASHION

Structured data wants lookups. Countable data wants aggregates. Prose wants hybrid search — keyword-leaning when queries quote exact terms, semantic-leaning when they describe symptoms. The fashionable answer (embed everything) is the OE of this domain; the stem's data shape is the tell that picks the machine.

⚠ TRAP · RAG AS A HAMMER

The 3.6 distractor of record: “index the TMS records so the model can semantically search shipment statuses.” The TMS already answers exactly; an index answers approximately. When a stem's data is rows with keys, the RAG option is wrong however well-designed its chunking is. Save your RAG craft for the 400 SOPs, where it is genuinely the right machine.

CHAPTER 7 · OBJECTIVE 3.3

The dials — accuracy and latency are bought, not wished

☂ THE EXPRESS LANE AND THE INSPECTION BAY

Every dock runs two speeds. The express lane waves routine trucks through in seconds — a glance, a stamp. The inspection bay takes twenty minutes and catches what the glance cannot. A dock that inspects everything jams; a dock that inspects nothing gets robbed. The skill is not choosing a speed — it is **routing each truck to the right speed and writing down why**.

Every dial below trades the same two currencies. The exam's 3.3 stems hand you a constraint (an SLA, a cost ceiling, an accuracy floor) and grade whether you move the RIGHT dial the RIGHT way — and can justify it in one sentence. The Kaveri settings, with their justifications:

Dial	Kaveri setting	The one-sentence justification
Model tier per route	Router: Haiku-class · drafts: Sonnet-class · refunds/agent tier: Opus-class	Spend capability where errors are expensive, not where volume is high
Retrieval depth (top-k)	k=20 after reranking for SOP answers; k=3 for quick lookups	Anthropic's data: 20 beat 5 and 10; but drafts that quote one section need 3
Rerank or not	ON for SOP answers (+~200 ms); OFF inside the 2 s router path	67% failure-rate cut is worth 200 ms everywhere the SLA can afford it
Context per call	Cached SOP preamble + top-k chunks; never the whole corpus on the hot path	Tokens are latency AND money; the cache pays only above the tier minimum
Timeouts + retries	Carrier APIs: 3 s timeout, one retry, then degrade to cached ETA	A hung integration must cost a known maximum, not an unknown wait
Parallel fan-out	Three carrier checks concurrently (D1 E4)	Wall clock = slowest scout; the SLA buys concurrency, not model speed

◎ THE TELL

3.3 stems bait the **TR** family: they fix one currency and offer an option that spends it to buy the other — “add reranking” when the stem fixed a 2-second SLA, “drop to a smaller model” when it fixed an accuracy floor on refunds. Underline the fixed currency first; every dial then has a legal direction.

⚠ TRAP · JUSTIFYING WITH ADJECTIVES

“A more capable model for better accuracy” is not a justification; it is a wish with a price tag. Exam-grade justification names the constraint, the dial, the direction, and the number: “the refund path has an accuracy floor and 9% of volume, so it gets the large model; blended cost stays under the ₹6 ceiling.” If your chosen option cannot be justified in that shape, it is probably the distractor.

CHAPTER 8 · OBJECTIVE 3.4

The dock cameras — observability at 40,000 a day

☂ CAMERAS, THE REGISTER, AND THE GAUGES

A dock that works fine is not a dock you can PROVE works fine. Three kinds of watching, three different questions: the **register** records what happened, one line per event (logs — “what did truck 4411 do?”); the **camera that follows one truck** gate to bay to gate (traces — “WHERE did truck 4411 lose forty minutes?”); the **gauges on the wall** — trucks per hour, average wait (metrics — “is today normal?”). Lose any one and a class of question becomes unanswerable. At 40,000 trucks, unanswerable means unfixable.

EXHIBIT E7 · THE CAMERAS, IN CODE

e7_dock_cameras.py

```python
import json, time, uuid, statistics as st

def log(**fields):                                # the register: one JSON line
    print(json.dumps(fields, ensure_ascii=False))

def handle(ticket):
    trace = str(uuid.uuid4())[:8]                 # the camera follows THIS truck
    t0 = time.monotonic()
    for stage, fn in [("route", route_it), ("retrieve", fetch_chunks),
                      ("draft", draft_reply)]:
        s0 = time.monotonic()
        out = fn(ticket)                          # each stage, timed alone
        log(trace=trace, stage=stage,
            ms=round((time.monotonic() - s0) * 1000),
            tokens=getattr(out, "tokens", 0),
            cost_pc=getattr(out, "paise", 0))     # paise, not vibes
    log(trace=trace, stage="total",
        ms=round((time.monotonic() - t0) * 1000))

def digest(lines):                                # the gauges, from the register
    by = {}
    for ln in map(json.loads, lines):
        by.setdefault(ln["stage"], []).append(ln["ms"])
    for stage, ms in by.items():
        ms.sort()
        p95 = ms[int(0.95 * len(ms)) - 1]
        print(f"{stage:9s} mean={st.mean(ms):7.0f}ms  p95={p95:7.0f}ms")
```

✎ PREDICT BEFORE RUN

Run the canned day (10,000 lines, provided). Predict: which stage owns most of the p95, and will any stage's mean look healthy while its p95 is awful? Write the two numbers you expect for 'retrieve' before looking.

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
route     mean=     94ms  p95=    180ms
retrieve  mean=    310ms  p95=  2,900ms   <- the mean lied
draft     mean=    880ms  p95=  1,400ms
total     mean=  1,290ms  p95=  4,100ms
# 5% of retrievals hit a cold carrier API. The mean hid them; the p95
# is 300 dispatchers' actual Tuesday. SLAs are promises about p95s.
```

DECODE · LINE BY LINE, ONE STORY

1–4	The story: **the dock gets its register.** Structured logging = one JSON line per event, machine-readable. Line 4's print stands in for a log shipper; the shape of the line is the lesson, not the plumbing.
6–8	The camera: one trace id minted per ticket and stamped on every line it touches. This id is what turns 40,000 interleaved lines back into individual journeys — drop it (tonight's sabotage) and the register becomes noise.
9–16	Every stage timed alone, and — line 16 — **costed alone**, in paise. Chapter 2's bloat audit and chapter 7's dials both read from exactly these fields; observability is what makes every other D3 argument numerical instead of rhetorical.
20–27	The gauges, computed FROM the register: sort each stage's times, take the mean and the 95th percentile. The output block is the whole objective in four lines: **a healthy mean and a sick p95 at the same stage**. Averages are where tail pain goes to hide, and SLAs live in the tail.

⚒ SABOTAGE LAB · BREAK EXACTLY ONE VARIABLE

Break line 7: mint the trace id INSIDE the stage loop instead. The register's line count is identical; predict what question just became unanswerable. Run digest() — notice it still works! — then try to reconstruct one slow ticket's journey. Lesson: **observability gaps don't error; they wait.**

Scale warning for the exam's “at scale” clause: metrics are cheap because they aggregate — but only if their label sets stay small. A per-consignment-id metric label is 40,000 new series a day (the **cardinality trap**): ids belong in *logs and traces*, categories belong in *metrics*. And alert on symptoms customers feel (p95, error rate, queue depth), not on every internal wobble — an alert channel that cries hourly is a channel nobody reads on the real bad day.

LAW 4 · YOU CANNOT FIX WHAT YOU CANNOT SEE

Logs answer WHAT, traces answer WHERE, metrics answer IS-THIS-NORMAL. Instrument stages, stamp a trace id at the door, report p95s not means, keep ids out of metric labels — and write the log line even when you deny a call. Every other chapter's numbers came from this one's plumbing.

⚠ TRAP · DASHBOARD THEATER

The SD ghost's D3 costume: “add a dashboard” to a stem whose system emits no trace ids and logs free-text strings. A dashboard renders what exists; it creates nothing. When the stem's gap is instrumentation, the option that adds panels is decoration — the correct option touches the emitting code. (Sibling costume: alerting on 100% of anomalies, which guarantees the one real page gets ignored.)

CHAPTER 9 · OBJECTIVE 3.8

The catalogue at the gate — progressive discovery vs the monolith

☂ HAND THE DRIVER THE CATALOGUE, NOT THE WAREHOUSE

A driver arrives for one pallet. Warehouse A loads his truck with everything he MIGHT need — he drives away slow, overweight, and mostly wrong. Warehouse B hands him a one-page catalogue at the gate; he asks for bay 12, gets bay 12. But note the fine print: for a tiny warehouse with three shelves that every driver uses every time, the catalogue is pure overhead — just load the three shelves. **The map first, the territory on demand — unless the territory is smaller than the map.**

This objective is where D3's economics all reconverge. “Monolithic context” = every tool schema, every SOP, every reference stuffed into each request. “Progressive discovery” = send a light index; fetch details when the task actually needs them. You have met both bills already: E2 priced schemas, E5 priced retrieval. E8 prices the strategy choice itself.

EXHIBIT E8 · TWO BILLS FOR THE SAME TASK

e8_two_bills.py

```python
# Task: answer one dispatcher question that needs 1 tool + 1 SOP section
MONO = {
    "tool_schemas": 9 * 260,          # every forklift's manual, every call
    "sop_corpus":   400 * 900,        # the whole warehouse, every call
    "question":     120,
}
PROG = {
    "catalogue":    9 * 25,           # one line per tool: name + when-to-use
    "fetch_tool":   260,              # full schema, fetched on demand
    "retrieved":    20 * 180,         # top-20 chunks, not 400 documents
    "question":     120,
    "extra_turn":   1,                # the fetch costs a round-trip
}
for name, bill in [("monolithic", MONO), ("progressive", PROG)]:
    toks = sum(v for k, v in bill.items() if k != "extra_turn")
    print(f"{name:12s}: {toks:8,d} tokens/call"
          f"{'  + 1 round-trip' if bill.get('extra_turn') else ''}")

# When the monolith WINS: corpus under ~200k tokens, stable, always-used
CACHED_SMALL = {"corpus": 45_000, "cache_hit_price": 0.1}  # 90% off after write
print(f"small+cached: {int(45_000 * 0.1):8,d} token-equivalents/call, no extra turn")
```

✎ PREDICT BEFORE RUN

Sum both bills in the notebook first — the gap is the point, get it in your hands. Then predict: at what corpus size and stability does the third line beat the second, and which two D3 facts (one from this chapter's DRIFT, one from caching) decide it?

Prediction, in the notebook, before running: .....

.....

EXPECTED OUTPUT

```
monolithic   :  362,460 tokens/call
progressive  :    4,205 tokens/call  + 1 round-trip
small+cached:    4,500 token-equivalents/call, no extra turn
# 86x between the first two. The third line is the monolith done RIGHT:
# small stable corpus + prompt caching ~= progressive costs, zero latency tax.
```

DECODE · LINE BY LINE, ONE STORY

1–6	The story: **one question, three ways to pay for it.** The monolith's bill: nine full schemas (E2's numbers) plus the entire SOP corpus, on EVERY call, for a question that needs one tool and one section. 362k tokens — most models' entire window, spent on insurance.
7–13	The catalogue: one line per tool ("name + when to use"), full schema fetched only when chosen — this is exactly how deferred-tool systems and MCP's tools/list → tools/call shape work, and E5's top-20 replaces the 400-document dump. Price: one extra round-trip when a fetch happens. **Progressive discovery trades latency for context** — a trade, not a free lunch, which is why 3.8 says "evaluate", not "always".
15–18	Both bills printed. 86× is the kind of number that ends meetings — but the exam wants the NEXT line too.
20–23	The monolith's honest case: a corpus under ~200k tokens (the skip-RAG threshold from chapter 5's DRIFT), stable and always-relevant, parked behind prompt caching (cache reads ~90% off, above the tier's minimum cacheable size) — near-progressive cost, zero extra turns. Small, stable, always-used → stuff and cache. Large, changing, occasionally-used → catalogue and fetch.

⇄ DRIFT · CONTEXT IS NOW MANAGED SERVER-SIDE TOO

Workbook / common teaching: Common teaching: when the window fills, summarize old turns yourself client-side. **Current docs (checked Sep 2026):** Current API: **context editing** runs server-side (beta header context-management-2025-06-27): clear_tool_uses_20250919 clears oldest tool results on a trigger (default 100k input tokens; keep default 3 recent pairs; exclude_tools protects named tools), and clear_thinking_20251015 manages thinking blocks. Clearing can invalidate cached prefixes — clear_at_least exists to make the invalidation worth it — and the memory_20250818 tool lets the model save essentials to files before they are cleared. Your client keeps the full history; the trim happens before Claude reads it.

⇄ DRIFT · UPDATED 4 OCT 2026 — TWO CORRECTIONS TO THIS BOOK

Workbook / common teaching: First printing (28 Sep): E8's expected output showed 4,625 tokens/call and a 78× gap; the glossary and Memory Map gave cache minimums as 1,024–4,096.
Current docs (checked Sep 2026): E8's own line items sum to 9×25 + 260 + 20×180 + 120 = **4,205** tokens/call, a gap of ≈ **86×** (corrected above and on Atlas A7). Cache minimums per current docs: **512–4,096** — newest tier (Opus 5.5, Fable 5.1) 512; Sonnet 5 = 1,024; Haiku 4.5 = 4,096; Opus 4.5/4.6 = 4,096. Reads 0.1× standard, cheaper still on the newest tier. The exam may quote any tier: read the model name in the stem.

LAW 5 · SEND THE MAP FIRST, THE TERRITORY ON DEMAND

Light indexes up front; details fetched when chosen; retrieval instead of corpus-stuffing; context editing to shed exhaust in long runs. Exception, learned by heart: small + stable + always-used + cacheable → monolith WITH caching wins, because the fetch's round-trip is the one cost the catalogue can never refund.

⚠ TRAP · PROGRESSIVE EVERYTHING

3.8's inverted distractor: catalogue-and-fetch applied to three tools and a 40k-token policy pack that every single call uses. Now every request pays a discovery round-trip to avoid a context cost that caching had already reduced ~90%. The tell is small, stable, always-used — when you read those three properties together, the monolith-plus-cache is the correct answer and the fashionable one is not.

CHAPTER CLOSE — WHAT DAY C WILL TEST

Forty-five questions on Monday. New-territory objectives carry the extra weight, VG is the family to starve — one glossary pass tonight, one tomorrow. Underline the tell; name the family of every distractor you reject. The key stays sealed until Q45. — *H.*

PART TWO

Glossary + Architecture Atlas

the dock's dictionary, then A1–A7 · Day D redraws A3 from memory

THE LAYMAN GLOSSARY

New territory, in dock language first

Read once on Day A (star the strangers), once on Day B (recite from the analogy column), retire the stars on Day D. Each entry: plain words → the dock analogy → the exam-grade line. ☐ = not yet mine.

RETRIEVAL & RAG

☐ **chunking** — Cutting documents into retrievable pieces.
Shelving: one meaning, one box.
Splitting a corpus into units for indexing; strategy (fixed / semantic / **heading-aware** / overlap) sets retrieval's ceiling.

☐ **embedding** — A text turned into a list of numbers that captures its meaning.
The foreman's memory of an aisle.
A dense vector representing semantics; similar meanings land near each other in vector space.

☐ **vector index** — The database that stores embeddings for fast nearest-neighbour search.
The foreman, institutionalised.
A store answering "which chunks are nearest this query vector" at scale.

☐ **BM25 / keyword search** — Classic exact-word search with clever weighting.
The register: literal, synonym-blind.
Lexical ranking rewarding term frequency and rarity; first-class in current practice, not legacy.

☐ **hybrid search** — Keyword and semantic search fused into one ranking.
Ask both; trust where they agree.
Score fusion of BM25 + embeddings; each channel covers the other's blind queries.

☐ **reranking** — A second, smarter pass that re-sorts the shortlist.
The supervisor re-sorting the picks.
A cross-reading model scores query+chunk pairs; shape: retrieve ~150 → rerank → top-20; ~+200 ms for a 67% failure cut (with contextual retrieval).

☐ **contextual retrieval** — Each chunk gets a sentence saying where it's from, before indexing.
A label INSIDE every box.
Prepend chunk-situating context before embedding AND BM25; failure-rate cuts 35%→49%→67% as stages stack.

☐ **top_k** — How many chunks the retriever hands the model.
How many boxes reach the desk.
Candidate count passed onward; Anthropic's tests: 20 beat 5 and 10 for final answering.

☐ **recall vs precision** — Did we fetch all the right boxes vs only right boxes.
Missing the sofa vs bringing the whole aisle.
Retrieval-sense: recall = fraction of relevant chunks found; precision = fraction of found chunks relevant. Dials trade them.

☐ **context poisoning** — Bad retrieved text steering the answer.
A mislabelled box on the desk.
Irrelevant/adversarial retrieved content degrading generation; mitigations: better retrieval, provenance labels, scoped prompts.

AUTH & SECURITY

☐ **authN vs authZ** — Who you are vs what you may do.
The gate pass vs the manifest.
Authentication verifies identity; authorization grants actions. Exam stems hide authZ gaps behind working authN.

☐ **scope** — A named permission on a credential.
One bay listed on the manifest.
The unit of authZ (e.g. refund:write); checked in code before the tool body runs.

☐ **least privilege** — Every caller gets the minimum scopes that do the job.
No master keys on the keyring.
Design rule; its violation (wildcards, god-mode service accounts) is the OP ghost's Professional costume.

☐ **token (auth sense)** — A string proving identity/permissions — NOT an LLM token.
The gate pass itself.
Bearer credential on requests (OAuth-issued on Streamable HTTP). Same word, two exam meanings — disambiguate by context.

☐ **service account** — An identity for a machine, not a person.
The forklift's own gate pass.
Machine caller identity; per-service accounts keep audit trails honest; one shared account destroys them.

☐ **audit log** — Append-only record of who did what, when.
The gatehouse ledger.
Written on every sensitive call — including denials; the input to incident review and chapter 8's dashboards.

OBSERVABILITY

☐ **structured logging** — Log lines as data (JSON), not prose.
The register, machine-readable.
One event per line with fields; greppable, aggregatable; free-text logs cannot feed metrics.

☐ **trace / span** — One request's journey / one timed step inside it.
Following a single truck gate-to-gate.
A trace id stamps every event of one request; spans time each stage. Answers WHERE time went.

- ☐ **p50 / p95 / p99** — The median and the tails of latency.
  The queue as the unlucky see it.
  Percentiles; SLAs are promises about tails. A healthy mean with a sick p95 is the classic hidden failure.

- ☐ **cardinality** — How many distinct label values a metric has.
  One gauge per truck = a wall of gauges.
  High-cardinality labels (ids!) explode metric series; ids belong in logs/traces, categories in metrics.

MCP & CONTEXT

- ☐ **MCP server / tools / resources / prompts** — A program exposing actions, data and templates over one standard.
  The dock connector; forklifts, manifest rack, laminated cards.
  Server primitives: tools (act), resources (inform), prompts (template). Discovery via */list; naming mcp__server__tool.

- ☐ **transport: stdio vs Streamable HTTP** — Local pipes vs remote HTTP.
  Same-dock chute vs the highway.
  stdio = local process, OS-level identity; Streamable HTTP = remote, standard HTTP auth (OAuth recommended). SSE-as-transport is gone.

- ☐ **elicitation** — The server asks the user a question mid-task.
  The gate guard phoning the office.
  Current client primitive (sampling & logging are deprecated, spec 2026-07-28).

- ☐ **progressive disclosure / discovery** — Light index first; details fetched on demand.
  The catalogue at the gate.
  Deferred loading of schemas/content; trades an extra round-trip for a small standing context.

- ☐ **monolithic context** — Everything stuffed into every request.
  Loading the whole warehouse on the truck.
  All schemas + corpus every call; correct ONLY when small + stable + always-used + cached (corpus ≲ 200k tokens).

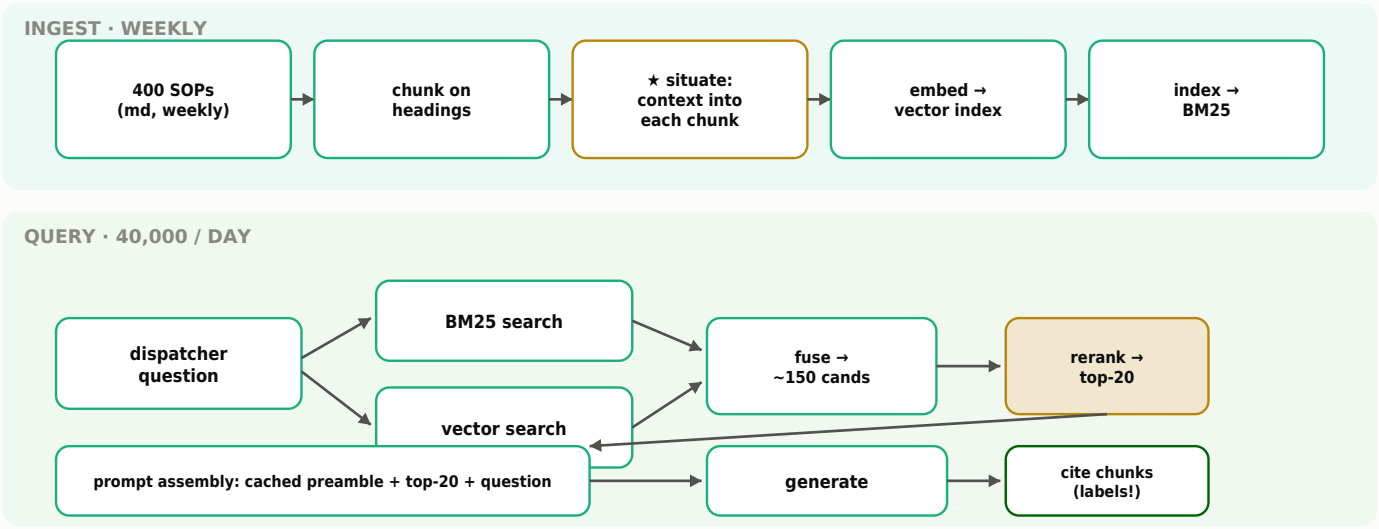
- ☐ **context editing** — The API trims old tool results server-side before Claude reads them.
  Sweeping the desk between jobs.
  clear_tool_uses_20250919 (trigger ~100k, keep 3, exclude_tools) + clear_thinking_20251015; clearing can invalidate cache — pair with clear_at_least and the memory tool.

- ☐ **prompt caching minimums** — Cached prefixes only pay off above a size floor.
  No discount on tiny parcels.
  Minimum cacheable prefix varies by tier (512–4,096 tokens: newest tier 512, Sonnet 5 = 1,024, Haiku 4.5 = 4,096); reads ~90% off (deeper on the newest tier). Re-verified 4 Oct 2026.

ATLAS A1

The Kaveri RAG pipeline — ingest lane, query lane, and the weekly loop

Two lanes. The top one runs weekly (the SOPs change); the bottom one runs 40,000 times a day. Contextual retrieval lives at the marked star: context is written into each chunk BEFORE both indexes.

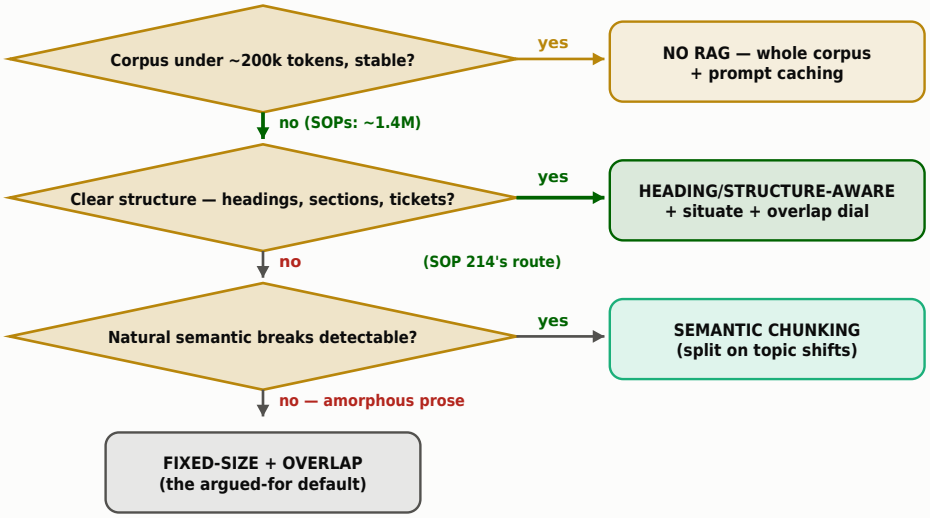


Numbers to keep: failure-rate cuts **35% → 49% → 67%** as situating, contextual BM25 and reranking stack · retrieve ~**150**, pass **20** · corpus ≲ **200k** tokens → skip this page entirely and cache the corpus.

ATLAS A2

The chunking-strategy picker — document shape in, strategy out

The 3.5 decision as a walk. SOP 214's route is drawn in green.



Overlay dials, every route: chunk size ≈ query granularity · overlap = seam insurance (duplicate-token cost) · situating context written into every chunk · re-chunk only when the miss pattern says the SHELVING failed (right doc, wrong section).

ATLAS A3

The retrieval-by-shape matrix — Day D redraws this from memory

Rows are what the data IS; columns are what the question ASKS. The crimson cells are the expensive mistakes; the exam's 3.6 stems live entirely on this grid.

Data shape ↓ · Query →	Exact entity ("where is KF-2231")	Procedure/policy prose ("how do I…")	Symptom/prose ("stuck, what now")	Count/trend ("how many late")
Rows with keys (TMS/WMS)	**Structured lookup** — the answer, not a guess	RAG here = OE — join the SOP corpus instead	Semantic search of rows = fluent nonsense	**SQL aggregate**
Structured docs (400 SOPs)	Keyword-weighted hybrid (exact codes, section numbers)	**Hybrid + rerank, top-20**	**Semantic-leaning hybrid** (synonyms carry it)	RAG cannot count — index metadata, aggregate that
Free-text logs / emails	Keyword (ids quoted verbatim)	Hybrid; situate chunks with ticket metadata	Semantic-leaning hybrid	Not retrieval — log pipeline → metrics (ch. 8)
Tiny stable reference (rate card, 6 templates)	No retrieval: in-context + cache	No retrieval: in-context + cache	Same	Trivial in code

◎ **THE TELL**

Read a 3.6 stem twice: once for the row (what the data IS), once for the column (what the question ASKS). An option that is excellent machinery in the wrong cell is still wrong — the grid outranks the craft.

ATLAS A4

The integration mechanism — MCP, direct API, or agent-to-agent

Three carrier integrations, three different right answers — drawn as the worked case.

MCP SERVER

BlueDart — rich API, four internal clients need it

- one connector, N clients
- discovery keeps clients current
- tool list can change safely
- remote: Streamable HTTP + OAuth

Hurts: Overhead never repaid when exactly one client will ever call.

DIRECT API / CLI

IndiaPost — one endpoint, one caller, tight budget

- least machinery
- no protocol layer to operate
- latency: one hop
- wrap it as an MCP tool LATER if clients multiply

Hurts: Becomes N custom welds the day a second client appears.

AGENT-TO-AGENT

Delhivery's booking agent — negotiates slots, has judgement

- exchange goals, not function calls
- structured handoff briefs (D1.4!)
- authorization state travels in the brief
- budget the conversation (D1 Law 3)

Hurts: Absurd for deterministic lookups — a phone call to ask the time.

The 3.7 sentence: count the CLIENTS and the CHURN. Many clients or a changing tool list → the standard connector earns its keep. One client, one owner, no churn → the weld. The far side has judgement → talk to it, with a briefed, budgeted conversation.

ATLAS A5

The auth boundary map — every door, every key

The control tower's doors as one table you could review in a meeting. The crimson row is the ghost as found on Saturday; the green row is Monday's fix.

Caller (identity)	authN — proves itself by	Scopes held (the manifest)	May therefore touch
KAVI agent (service acct)	API key, rotated	status:read · carriers:read · sop:read	TMS status, carrier lists, SOP store
Refund desk service	OAuth token (Streamable HTTP)	status:read · refund:write	process_refund — and the ₹10k human gate still applies above threshold (D5)
Dispatcher UI	SSO session	status:read · draft:write	read statuses, edit drafts — never refunds
carrier-portal-v1 (AS FOUND)	static token, 18 months old	**status:read · * (wildcard)**	**everything — every customer's consignments, refunds included**
carrier-portal-v1 (FIXED)	OAuth, 90-day rotation	**status:read only**	**statuses on its own lane, nothing else**
Observability shipper	mTLS service identity	logs:write	append to the log store — cannot read business data

Rules the map encodes: per-caller identities (no shared god account) · every tool names its required scope in code · denials are logged and dashboarded · wildcards are findings, not conveniences · the gate check runs before the tool body, always.

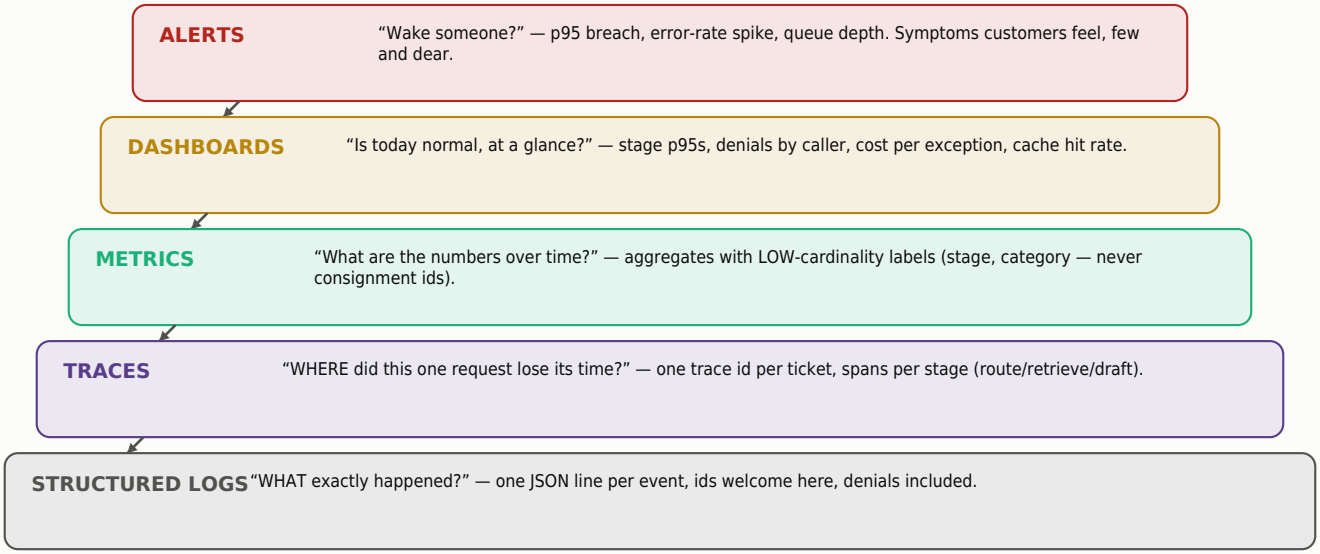
⚠ TRAP · THE MAP NOBODY DREW

3.2's meta-lesson: the heist survives in systems where this table exists only in people's heads. An exam option that says “document and review per-caller scopes” beats clever cryptography options whenever the stem shows a working login and an unexplained data leak.

ATLAS A6

The observability stack at 40,000/day — what question each layer answers

Bottom feeds top. Every layer exists to answer one kind of question; an incident walks DOWN the stack: alert → dashboard → metric → trace → log line.

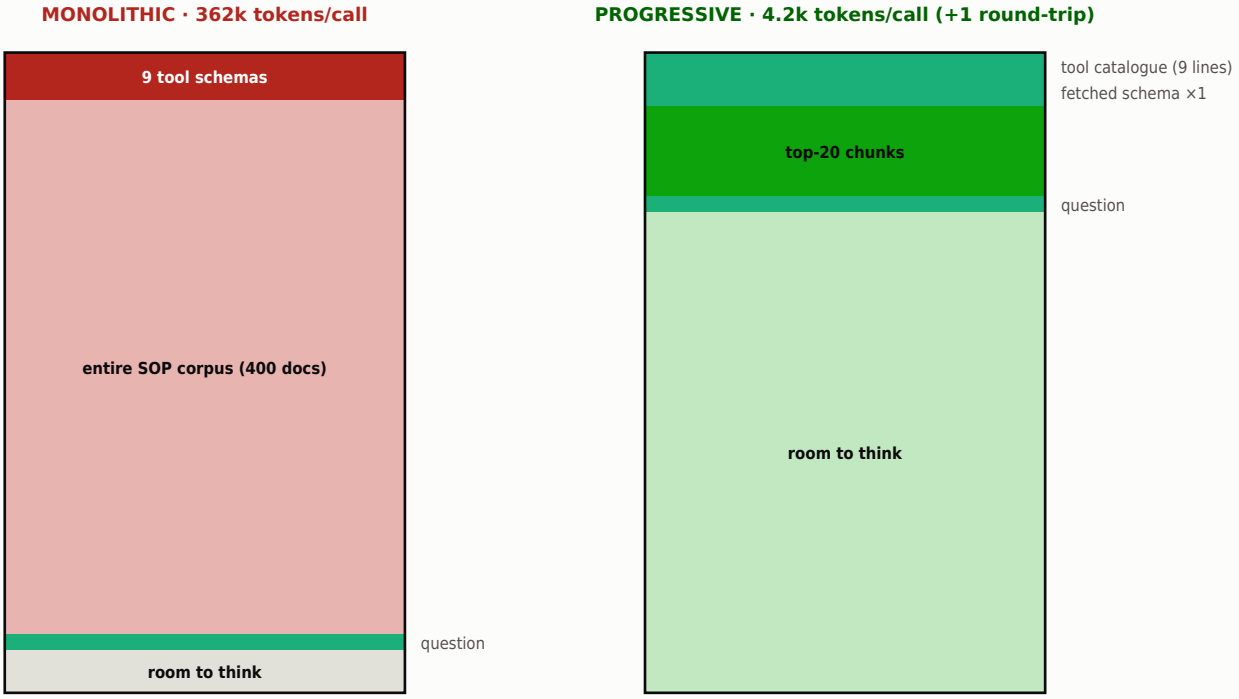


The cardinality trap, drawn: a metric labelled by consignment id mints 40,000 series a day and melts the metrics store. Ids live in the two bottom layers; the top three see categories. **The incident walk:** page fires (p95) → dashboard names the stage (retrieve) → metric shows since when → trace shows the slow span (carrier API) → log line has the request. Five layers, one minute, no guessing.

ATLAS A7

Two context strategies, drawn as the window they fill

Same task, both strategies, to scale. The bar is the model's context window; what you fill it with is what you pay for — in tokens, and in the model's attention.



The third bar you cannot draw the same way: small + stable + always-used corpus (≲ 200k tokens) behind prompt caching — the monolith's shape at near-progressive cost, with **no** fetch round-trip. And for long agent runs, **context editing** empties the “old tool results” shelf server-side as the window fills (trigger ~100k, keep 3), trading a cache re-write for headroom.

⚠ TRAP · READING ONLY THE TOKEN COLUMN

The monolith's real cost is not only the bill — it is the thin grey “room to think” slice, and retrieval quality degrading as the window fills with mostly-irrelevant text. The progressive bar's real cost is the round-trip and the risk of under-fetching. 3.8 options that mention only one side's cost are advertising, not analysis.

PART THREE

The Dock Book — 45 questions

Sat cold on Day C · one sitting · ~90 min · key sealed until Q45 · underline the tell before the options

PART THREE

The Dock Book — instructions, then forty-five questions

Distribution: 3.1×5 · 3.2×6 · 3.3×5 · 3.4×6 · 3.5×7 · 3.6×6 · 3.7×5 · 3.8×5 — new territory carries the weight. Fourteen head-to-head pairs; ten number-decided stems; four D1 crossovers, because the real form mixes domains. Rules: underline the deciding clause first; 120 seconds each; key sealed; every miss gets a ledger row with a family code.

1 KAVI's config lists nine tools. Last week's logs: three tools carried 99.6% of calls; five were never called; escalate_to_human was called twice, both on refund disputes above ₹10,000. Token count shows 2,314 tokens of schema per call versus 811 for a trimmed set. The 3.1-correct trim: — objective 3.1

A. Trim to one general-purpose tool that wraps the rest
B. Keep the three plus escalate_to_human; re-add others when a log line asks
C. Keep only the three tools with traffic; escalation can be re-added later if a dispatcher ever asks for it
D. Keep all nine — capability is cheap insurance

2 Two tools survive an audit: lookup_order and get_shipment_status. Logs show the model calls the wrong one in 7% of cases; their descriptions overlap heavily. Best 3.1 fix: — objective 3.1

A. Add a third tool whose only job is to decide which of the two existing tools the model should call
B. Merge them, or rewrite both descriptions until the boundary is unambiguous
C. Route all calls to whichever tool is cheaper
D. Fine-tune the model on correct tool choices

3 A stem proposes adding six “likely useful someday” tools before launch, arguing schemas are small. At 40,000 calls/day, 250 tokens per schema, what is the daily cost of that argument, and the verdict? — objective 3.1

A. ~0.6M tokens/day; acceptable insurance
B. Token cost is irrelevant at this scale; reject the extra tools only if the security review requires it
C. ~6M tokens/day; accept if cached
D. ~60M tokens/day; reject — pay for capability the day a log line asks for it

4 Which situation justifies ADDING a tool this sprint? — objective 3.1

A. The demo would look stronger with more capabilities
B. A competitor's agent reportedly has twelve tools
C. The TMS vendor shipped a new API this sprint and integration is trivial, so adding the tool costs almost nothing
D. Dispatcher tickets show 40 cases/week manually looking up customs duty rates the model cannot reach

5 KAVI at the agent tier sometimes receives 30+ tools when MCP servers are aggregated. The team proposes exposing every server's full tool list to every request. The measured effect: schema bill triples and wrong-tool calls rise. The 3.1/3.8 answer: — objective 3.1

A. Expose a per-route subset (the agent tier's own belt), with the rest discoverable on demand
B. Drop MCP aggregation altogether and hardcode the three most-used tools directly into the agent tier's code
C. Ask the model to ignore irrelevant tools
D. Accept; aggregation is MCP's point

6 Every integration authenticates flawlessly. During an audit Arjun finds the carrier-portal token, issued 18 months ago, carries a wildcard scope. Nothing bad has happened. The 3.2-correct reading: — objective 3.2

A. Add a system-prompt rule forbidding any misuse of the carrier portal, and log every call the model makes to it for review
B. A live security gap: valid authN with over-broad authZ is exactly how the Refund Heist worked — narrow it now
C. Rotate the token; rotation fixes staleness
D. No incident, no problem; schedule a review next quarter

7 Where must the check 'refund-desk may call process_refund; nobody else may' live? — objective 3.2

A. In the model's judgement, given good context about who the caller is and what refunds are allowed
B. In the system prompt, stated firmly
C. In weekly log review
D. In code, before the tool body runs, keyed on the caller's scopes

8 The team runs every tool call through one shared service account “to simplify ops.” Which capability dies, and why does the exam care? — objective 3.2

A. Latency budgeting; extra hops
B. Nothing dies; one well-guarded service account is simpler to rotate and monitor than a dozen separate identities, which is good ops
C. Caching; accounts affect cache keys
D. Attribution: the audit log can no longer say WHO did what, so authZ cannot be scoped per caller and incidents cannot be traced

9 A remote MCP server for the SOP store must be exposed to three internal teams over Streamable HTTP. The 3.2-grade authN choice: — objective 3.2

A. A single long-lived API key shared by the three teams, rotated every quarter by the platform team
B. OAuth-issued tokens per team, scoped sop:read, with rotation
C. IP allowlisting alone; the network is trusted
D. No auth; SOPs are internal documents anyway

10 Head-to-head. The stem: “dispatcher UI users must never trigger refunds, even if the model is convinced.” Plan P: omit refund tools from the UI's tool set and deny refund:write to its identity; Plan Q: instruct the model that dispatcher sessions must not refund, and log attempts. Pick and justify. — objective 3.2

A. Either; both prevent refunds
B. Plan P — capability absent and scope denied: the action is impossible, not discouraged
C. Plan Q, but with a louder instruction and alerts
D. Plan Q — instructions plus logging is defence in depth, and the audit trail catches anything the instruction misses

11 Which log discipline turns the gate pass system into a detection system? — objective 3.2

A. Log successful calls only, since denials are noise that would swamp the dashboard and hide the real traffic patterns
B. Log nothing sensitive; privacy first
C. Log every authorization decision including denials, and dashboard denials by caller
D. Log full request bodies for everything

12 The router path carries a 2-second end-to-end SLA. An option proposes adding reranking (+200 ms, better SOP answers) inside that path. Current p95 is 1.9 s. Verdict: — objective 3.3

A. Reject on this path: 1.9 s + 0.2 s breaches the SLA at p95; apply reranking on the non-SLA SOP path instead
B. Add it and raise the SLA
C. Reject reranking everywhere in the tower; an extra 200 ms per answer is too slow for a dispatcher-facing system on any path
D. Add it — accuracy always wins

13 Refund recommendations (9% of volume) show error costs ~₹1,800 per mistake; delay drafts (55%) cost ~₹4 per mistake. A single mid-tier model currently serves both. The justified 3.3 move: — objective 3.3

A. Keep one mid-tier model for both paths; the simplicity of a single configuration outranks any gain from tuning by category
B. Upgrade everything to the largest model
C. Split: large model on refunds, small on delay drafts — spend capability where errors are expensive
D. Downgrade everything to control cost

14 Which justification is exam-grade for setting top_k=20 on the SOP path? — objective 3.3

A. Twenty feels comprehensive
B. The vector DB defaults to 20
C. More chunks are always better, because the model can ignore the irrelevant ones and the extra context costs only a few tokens
D. Measured: top-20 outperformed top-5 and top-10 on answer quality; the token cost fits the path's budget

15 A carrier ETA API hangs occasionally. Currently: no timeout, calls wait up to 40 s. The dispatcher-facing SLA is 5 s. The 3.3-correct configuration: — objective 3.3

A. Move the ETA calls to a bigger model, whose stronger reasoning will cope better with the carrier's slow and inconsistent replies
B. Drop the carrier integration
C. Retry three times with backoff
D. 3 s timeout, one retry, then degrade to the cached last-known ETA with a staleness label

16 Head-to-head under a ₹6/exception blended cost ceiling: Plan P: cache the 8,000-token SOP preamble on the drafting path (hit rate ~92%); Plan Q: shorten the preamble to 900 tokens by summarising it. Measured: caching lands blended cost at ₹4.90 with unchanged accuracy; the summary lands ₹4.60 but drops draft accuracy 3 points. Pick. *objective 3.3*

A. Neither; renegotiate the ceiling
B. Plan P — both fit the ceiling; only Plan P holds accuracy, so the ceiling stops deciding and accuracy does
C. Plan Q — cheapest wins
D. Plan Q — the 3-point accuracy drop is negligible against a saving that recurs on every one of the 40,000 daily calls

17 Retrieval mean latency is 310 ms; p95 is 2.9 s; the SLA promises 3 s end-to-end. Leadership sees the mean and declares health. The 3.4-correct statement: *objective 3.4*

A. The system is healthy; the mean is well inside budget
B. Raise the SLA to match the mean
C. 5% of dispatchers live at 2.9 s in retrieval alone — the SLA is at risk at p95, and SLAs are promises about tails
D. Means and percentiles measure the same underlying latency at different sample sizes, so either one reports the system's health

18 40,000 interleaved log lines per day. Which single missing field makes the question “where did ticket T lose its time?” unanswerable? *objective 3.4*

A. The model version
B. The caller's scope list, without which the slow stage cannot be tied to the identity that triggered it
C. The token count
D. A per-request trace id stamped on every stage's line

19 A proposal: emit a latency metric labelled by consignment id “for maximum drill-down.” The 3.4 verdict: *objective 3.4*

A. Approve but sample 10% of ids
B. Approve; more labels, more insight
C. Reject: 40,000 new series/day — the cardinality trap; ids belong in logs and traces, categories in metrics
D. Reject metrics entirely; the structured logs already hold every id and timing, so they suffice for any question the team will ask

20 The alert channel fires ~30 times a day on internal wobbles; last month a real p95 breach scrolled past unnoticed for 6 hours. The 3.4 fix: *objective 3.4*

A. Add more alerts so nothing is missed
B. Silence the channel during business hours, when the dispatchers are watching the floor anyway, and let it fire only at night
C. Alert only on customer-felt symptoms (p95, error rate, queue depth); demote wobbles to dashboards
D. Route alerts to more people

21 Which pairing correctly matches question to layer? *objective 3.4*

A. All three questions → the dashboard
B. 'What exactly happened on call X?' → metrics with the call id as a label; 'Is today normal?' → the structured logs, read in order
C. 'What exactly happened?' → structured logs; 'Where did the time go?' → traces; 'Is today normal?' → metrics
D. 'Where did the time go?' → alerts

22 The denials-by-caller dashboard (built in ch. 3) spikes for carrier-portal-v1 at 03:00. Latency and error rates are normal. The 3.4/3.2-correct reading: *objective 3.4*

A. Restart the portal integration
B. Noise; nothing customer-facing degraded
C. A possible probe against narrowed scopes — investigate the trace ids behind the denials before anything degrades
D. The scope narrowing was too strict for the carrier's overnight batch jobs; widen it until the denials stop appearing

23 The SOPs are heading-structured, updated weekly, queried at section granularity. The 3.5-correct chunking: *objective 3.5*

A. One chunk per document; context is king
B. Fixed 500-token chunks with a 50-token overlap; uniform chunks are efficient to index and simple to maintain week to week
C. Sentence-level chunks for precision
D. Heading-aware chunks with the section title written into each chunk's text, modest overlap

24 Retrieval for “e-way bill re-filing” returns the CANCELLATION section. Both sections share vocabulary. Chunks were cut fixed-size mid-section. Where is the fix? — objective 3.5

A. Add a system-prompt warning that cancellation and re-filing are different procedures and must never be confused in an answer

B. Re-chunk on headings so each section is one self-labelled unit — the failure happened at ingest

C. Swap the embedding model

D. Raise top_k so both sections come back

25 What does contextual retrieval add over plain heading-aware chunking, and what does it cost? — objective 3.5

A. Automatic re-chunking on every query

B. A model-written situating sentence per chunk before embedding AND BM25 — one-time ingest cost (cacheable), for a 35–49% retrieval-failure cut before reranking

C. A second embedding model run at query time alongside the first, doubling query cost but catching the paragraphs the first one misses

D. Nothing measurable; it is branding

26 The corpus is the carrier rate card plus 6 SOP templates: 38k tokens, changes quarterly, needed on almost every call. The 3.5-correct architecture: — objective 3.5

A. Fine-tune the model on the rate card

B. No RAG: include it all in the prompt behind prompt caching

C. A full RAG pipeline with hybrid retrieval and reranking, so the design scales when the corpus grows

D. A vector index over the templates only

27 Weekly SOP updates arrive. Which pipeline property is REQUIRED for retrieval to stay truthful? — objective 3.5

A. A scheduled re-chunk + re-index run, with version labels inside chunks so stale citations are visible

B. Nothing; embeddings age gracefully

C. Real-time re-embedding on every edit

D. Doubling top_k so that enough chunks come back for the current version of a procedure to appear among the stale ones

28 Overlap of 40 tokens is proposed for the heading-aware chunks. The honest accounting: — objective 3.5

A. Seam-straddling sentences survive in one whole copy; cost = duplicate tokens indexed and sometimes retrieved twice — a dial, tuned per corpus

B. Overlap replaces the need for situating context

C. Overlap is free insurance

D. Overlap fixes the vocabulary confusion between neighbouring sections, because each chunk now carries some of the other section's words too

29 Head-to-head for the 400-SOP corpus: Plan P: contextual chunks + hybrid + rerank (answer p95 +200 ms, failure rate 1.9%); Plan Q: plain chunks + vector-only (p95 unchanged, failure rate 5.7%). The stem's only constraint: dispatcher answers must cite the correct section ≥ 97% of the time; latency budget 4 s, current p95 2.8 s. Pick. — objective 3.5

A. Plan P — 98.1% ≥ 97% while Plan Q's 94.3% fails the floor; +200 ms fits inside 4 s

B. Plan Q — never add latency

C. Plan Q, with top_k raised to compensate

D. Plan P, but only if latency is removed from the SLA, since 200 ms on every answer is a cost the dispatchers will feel

30 “Where is KF-2481 right now?” The system semantically searches an index of TMS records and answers from the most similar chunks. The 3.6 verdict: — objective 3.6

A. Acceptable if recall is tuned high

B. Good design; everything benefits from embeddings, and similarity search will still find the right record on almost every query

C. Wrong machine: keyed rows answer exactly via lookup; similarity search converts certainty into probability

D. Acceptable if the index refreshes hourly

31 “How many shipments ran late this week?” routed to hybrid RAG returns a fluent, cited-looking number. What actually happened, and the fix? — objective 3.6

A. The RAG counted the rows; fine

B. The embedding model is weak at numbers, so the chunks about late shipments were ranked badly; a model trained on numeric text would fix it

C. Chunks MENTIONING lateness were retrieved and the model composed a plausible number — route count/trend language to an aggregate query instead

D. top_k was too low for counting

32 Queries quoting exact codes (“HSN 8471”, “KF-”, section numbers) underperform on the semantic-heavy hybrid. The 3.6 dial: — objective 3.6

A. Lean the fusion toward keyword/BM25 for code-bearing queries — exact strings are the register's home game
B. Strip the codes out of queries before searching, so the semantic channel is not confused by strings that carry no meaning
C. Raise semantic weight further
D. Lower top_k

33 “Customer says the parcel is stuck and she's furious, what do I do” retrieves poorly on keyword search alone. Why, and the fix? — objective 3.6

A. No lexical overlap: SOPs say 'customs hold' and 'escalation', not 'stuck' and 'furious' — the semantic channel bridges the vocabulary gap
B. The corpus lacks relevant SOPs
C. Add the words 'stuck' and 'furious' to the relevant SOPs, so that keyword search can match the way customers actually talk
D. The query is too long for BM25

34 Free-text carrier emails must become searchable for dispute resolution. The 3.6-grade ingest: — objective 3.6

A. Summarise each email with a small model and index only the summaries, which are shorter, cleaner and cheaper to embed
B. Embed raw emails as-is
C. Keyword-index only; emails are informal
D. Situate each email chunk with metadata (carrier, consignment, date, thread) written into the text, then hybrid-index

35 A D1 crossover. The retrieval router (E6) misclassifies ~2% of aggregate-language queries into hybrid RAG. The team proposes replacing the router with an agent that reasons about each query. Verdict: — objective 3.6

A. Run both and compare vibes
B. Keep the router; fix its rule set or classifier with the 2% as an eval set — a known-category decision is workflow territory (D1 Law 2)
C. Adopt; agents route better
D. Route everything to hybrid RAG for simplicity, and let the model decide for itself when an answer needs a count rather than a passage

36 Four internal clients (KAVI, control tower, Arjun's IDE, the dispatcher UI) need the same shipment tools, and the tool list grows monthly. The 3.7-correct mechanism: — objective 3.7

A. One MCP server; clients discover tools via tools/list, so additions propagate without client changes
B. Agent-to-agent between the clients
C. Each client integrates the TMS API directly
D. A shared Python library, vendored into each of the four clients and bumped whenever the tool list changes

37 A tool must let clients read SOP 214's full text without spending a tool call. Which MCP primitive, and why? — objective 3.7

A. A resource (sop://214): data-not-action, readable into context via resources/read
B. A tool returning the file contents
C. A prompt embedding the SOP text
D. An elicitation request that returns the document text to the client as its answer, so no tool call is spent

38 A workbook states MCP remote servers use the SSE transport and that servers commonly call the client's LLM via sampling. Against the current spec, the accurate correction: — objective 3.7

A. Both were removed from the protocol years ago; the workbook invented them, and neither ever appeared in a published spec
B. SSE is current; sampling is deprecated
C. Both statements are current
D. Transports are stdio and Streamable HTTP; sampling (and logging) are deprecated — integrate the LLM directly, log to stderr/OTel

39 IndiaPost offers one REST endpoint; exactly one internal caller will ever use it; the latency budget is tight. The 3.7 choice: — objective 3.7

A. Call the API directly in code; wrap it as an MCP tool only if a second client appears
B. Stand up an MCP server around it now
C. Refuse the integration as non-standard until IndiaPost publishes an MCP server, since one-off welds are what the platform team banned
D. Agent-to-agent with IndiaPost

40 Delhivery exposes a booking AGENT that negotiates pickup slots given constraints. Integrating it correctly means: *objective 3.7*

A. Agent-to-agent: send a structured brief (consignment, constraints, authority limits), budget the exchange, and treat replies as proposals to verify

B. Screen-scraping its UI

C. Wrapping it as a stateless tool and calling it like a lookup, with the constraints passed as arguments and the reply parsed as the booking

D. Asking Delhivery for database access instead

41 30 tools across aggregated MCP servers; any single request uses at most 3. Schemas cost ~260 tokens each. The 3.8-correct context strategy: *objective 3.8*

A. Cache the 30 schemas and stop thinking about it

B. A one-line catalogue of all 30; full schemas fetched on demand — ~7.8k standing tokens become ~1.5k

C. All 30 schemas on every call, cached behind a breakpoint; completeness first, and the cache absorbs most of the token bill

D. Drop to 3 tools permanently

42 A 45k-token policy pack is stable for quarters and consulted by essentially every call. Progressive fetching is proposed “as best practice.” Verdict: *objective 3.8*

A. Reject both caching and fetching; paste a 2,000-token summary of the policy pack into every call instead of the full text

B. Adopt half: fetch the pack every second call

C. Reject: small + stable + always-used → monolith WITH prompt caching; the fetch round-trip is pure loss here

D. Adopt; progressive is always leaner

43 An agent-tier investigation runs 40 turns; old tool results push input past 150k tokens and costs climb. The current-API answer: *objective 3.8*

A. Summarise the conversation manually on the client each turn and send the summary in place of the old tool results

B. Restart the conversation every 10 turns

C. Raise max_tokens

D. Server-side context editing: clear_tool_uses on a token trigger, keep recent pairs, exclude critical tools; pair with the memory tool so essentials survive clearing

44 Context editing clears 60k tokens of old tool results mid-run. What is the honest cost accounting? *objective 3.8*

A. The cleared prefix invalidates cache at that point — one cache re-write, then cheaper turns; clear_at_least exists to make the trade worthwhile

B. Free; cleared tokens were waste

C. Clearing corrupts the model's memory of its own reasoning, so later turns contradict earlier ones and the investigation loses its thread

D. It doubles latency permanently

45 Head-to-head, D1 crossover. A nightly batch agent (E8-style, 400 cases) needs the SOP corpus. Plan P: monolithic corpus in every case's context; Plan Q: per-case hybrid retrieval, top-20. Numbers: corpus 1.4M tokens (over any window); retrieval adds 180 ms/case; the batch window is 4 hours. Pick. *objective 3.8*

A. Neither; skip the SOPs at night

B. Plan P — batches prefer simplicity

C. Plan Q — 1.4M tokens cannot ride in any context; retrieval's 180 ms × 400 = 72 s against a 4-hour window

D. Plan P — with the corpus split across several turns so that each case still sees every SOP before it is decided

SEALED

THE KEY

Do not turn this page until all forty-five answers are marked.

Score by objective. 34+/45 with no objective below 60% = on track; a VG-heavy miss profile = one more glossary pass.

THE KEY

Answers, tells, and why each distractor fails

Families: **CM** Constraint missed · **HL** Human loop skipped · **OE** Over-engineered · **OP** Over-permissive · **SD** Sounds-diligent · **SF** Silent failure · **TR** Trade-off reversed · **VG** Vocabulary gap.

Q1 **B** — obj 3.1

The tell: Usage counts AND consequence decide: escalation ran twice, on the worst days.
Why B: Trim by logs and consequence together; the belt keeps rare-but-critical tools.
A [OE/VG] a mega-tool hides nine schemas inside one worse one
C [CM] retires the tool whose rare use is its entire value
D [OE] 1,500 tokens/call of insurance × 40,000 calls/day, plus lookalike confusion

Q2 **B** — obj 3.1

The tell: Lookalikes raise wrong-tool rates; the fix is the belt, not more belt.
Why B: Bloat includes indistinct tools. Fewer, sharper doors beat a doorman.
A [OE] a coordinator over two tools; S3's ghost at tool scale
C [TR] cost dial moved when the broken dial is distinctness
D [OE] training-scale machinery for a description-scale defect

Q3 **D** — obj 3.1

The tell: 6 × 250 × 40,000 = 60M — arithmetic, not opinion.
Why D: Do the multiplication in the margin; 'someday' tools bill daily.
A [VG] slipped two orders of magnitude
B [CM] cost is exactly what 3.1 grades, alongside confusion
C [CM] right order? No: 6×250×40k = 60M, and caching discounts, not erases; still the wrong default

Q4 **D** — obj 3.1

The tell: A logged, recurring need the current belt cannot serve — the only door-opening argument.
Why D: The anti-over-learning question: doors open when logs ask, and this log asks weekly.
A [SD] demos are not workloads
B [SD] competitor counts are not constraints
C [OE] ease of adding is not need

Q5 **A** — obj 3.1

The tell: Per-route subset + discoverable rest — the belt and the catalogue, combined.
Why A: Bloat control at fleet scale: scope the default set, defer the tail.
B [TR] throws away multi-client benefits to fix a config problem
C [SD] instructions don't unload a belt; config does
D [CM] aggregation is transport, not a reason to wear every belt at once

Q6 **B** — obj 3.2

The tell: Valid credential, too many bays — the gap is authZ, and it is live today.
Why B: AuthN can be perfect while authZ is the hole. Narrow scopes, then rotate too.
A [SD] persuasion guarding a door config left open
C [CM] a fresh key to every bay is still a key to every bay
D [SF] the incident is the discovery; waiting prices the next heist

Q7 **D** — obj 3.2

The tell: "Nobody else may" is a guarantee; guarantees live in code.
Why D: Config beats instruction — authorization is its Professional name.
A [SD] judgement is not a control
B [SD] the persuasion gate; CCAR-F's core discriminator
C [SF] detects Tuesday's breach on Friday

Q8 **D** — obj 3.2

The tell: One identity for all callers = a register that reads 'somebody' on every line.
Why D: Per-caller identity is the precondition for scopes, audits, and honest dashboards.
A [VG] unrelated mechanism
B [CM] the simplification deletes least privilege AND auditability
C [VG] unrelated mechanism

Q9 **B** — obj 3.2

The tell: Remote transport + multiple callers → standard HTTP auth, OAuth recommended, scoped per caller.
Why B: Transport shapes authN: stdio inherits the OS user; Streamable HTTP wants OAuth + scopes.
A [OP-shaped OE-inverse] shared key = no attribution, no narrow scopes
C [SD] networks move; identity is not an IP
D [CM] 'internal' data still feeds prompts; poisoned or exfiltrated context is a real gap

Q10 **B** — obj 3.2

The tell: "Even if the model is convinced" — the stem explicitly defeats persuasion-based options.
Why B: The tell names model conviction, which only config survives. Tool-list omission + scope denial.
A [CM] one prevents, one requests
C [SD] volume is not enforcement
D [SD] logging an attempt is not preventing it

Q11 **C** — obj 3.2

The tell: Denials by caller spiking is a probe in progress — but only if written down.
Why C: The audit log is a sensor. Denied calls are its most interesting lines.
A [SF] the probe pattern lives in the denials
B [CM] decision metadata isn't payload; auditability is a security control
D [OP] payload hoarding creates a new leak to guard

Q12 **A** — obj 3.3

The tell: 1.9 + 0.2 > 2.0 at p95 — one addition, one breach.
Why A: Dials are per-path. The same upgrade is right on one route and wrong on another.
B [CM] the SLA is the contract, not the variable
C [TR-inverse] rejects a 67%-failure-cut dial where latency is slack
D [TR] spends the currency the stem fixed

Q13 **C** — obj 3.3

The tell: ₹1,800 vs ₹4 per error — the asymmetry IS the answer.
Why C: Tiering by consequence is the 3.3 signature move — justify with both numbers.
A [SD] 'simplicity' with a number against it is a preference, not a justification
B [OE] pays refund-grade prices for template apologies
D [TR] saves paise on the path that loses thousands

Q14 **D** — obj 3.3

The tell: Measured, plus fits the budget — number, source, constraint.
Why D: 3.3 answers have the shape: metric moved, constraint respected, sentence of why.
A [VG] vibes
B [SD] defaults are not decisions
C [TR] past some k, noise displaces signal and cost rises for nothing

Q15 **D** — obj 3.3

The tell: A hung dependency must cost a known maximum under a 5 s SLA.
Why D: Timeout + bounded retry + honest degradation: latency engineering, D3-style.
A [VG] model size does not time out an API
B [OE-inverse retreat] the data is valuable; the wait is the problem
C [CM] three retries of a hang can exceed 40 s; no cap on the wait

Q16 **B** — obj 3.3

The tell: Both clear ₹6 — so the binding constraint moves to the next dial, accuracy.
Why B: When two options both meet the stated constraint, the tie-break is the currency the stem did NOT free up.
A [SD] nothing needs renegotiating; an option satisfies everything
C [TR] optimises past the ceiling into a currency the stem didn't discount
D [CM] the stem gave no licence to spend accuracy

Q17 **C** — obj 3.4

The tell: Mean 310 / p95 2,900 — the tail is where the promise breaks.
Why C: Report p95s. The exam's observability stems almost always hinge on a tail.
A [SF] the mean is where tail pain hides
B [CM] the SLA is the contract
D [VG] they measure different things by construction

Q18 **D** — obj 3.4

The tell: Reconstructing one journey from interleaved lines requires the id that threads them.
Why D: Logs answer WHAT; only a trace id turns them into WHERE.
A [CM] useful, answers other questions
B [CM] chapter 3's question, not this one
C [CM] cost questions, not journey questions

Q19 **C** — obj 3.4

The tell: Id as a metric label = a gauge per truck, forever.
Why C: Cardinality discipline: id → log/trace; category → metric.
A [CM] 4,000 series/day is still the trap, slower
B [OE/VG] melts the metrics store
D [TR] throws away trends to dodge a labelling mistake

Q20 **C** — obj 3.4

The tell: A channel that cries thirty times a day has trained its readers to scroll.
Why C: Symptom-based alerting: few, dear, actionable. Everything else is a panel.
A [OE] more noise is not more signal
B [SF] institutionalised blindness
D [SD] wider distribution of noise is still noise

Q21 **C** — obj 3.4

The tell: Three questions, three layers — the A6 stack verbatim.
Why C: WHAT/WHERE/NORMAL → logs/traces/metrics. Recite it.
A [SD] dashboards render layers; they are not the layers
B [VG] reversed
D [VG] alerts are triggered symptoms, not diagnosis

Q22 **C** — obj 3.4

The tell: Denials spiking with healthy latency is a security signal, not a performance one.
Why C: Observability serves security too: the denial panel exists for exactly this 03:00.
A [SD] restarts cure state, not intent
B [SF] 'nothing degraded' is the attacker's favourite dashboard
D [OP] widening scopes because someone is knocking is opening the door

Q23 **D** — obj 3.5

The tell: Structure exists and queries are section-shaped — cut on the seams.
Why D: Chunk granularity mirrors query granularity; origin labels ride inside the text.
A [TR] 900-token answers to 40-token questions; retrieval precision dies
B [CM] meaning-blind cuts split procedures (the KF-2481 incident)
C [CM] granularity finer than any query; procedures shredded

Q24 **B** — obj 3.5

The tell: Cut mid-section — the ceiling was set at shelving time.
Why B: Diagnose by pipeline stage: this miss pattern names ingest, so fix ingest.
A [SD] persuasion downstream of a structural defect
C [TR] a better foreman cannot find a box that was sawn in half
D [SF] ships the confusion to the model and hopes

Q25 **B** — obj 3.5

The tell: Before embedding AND BM25, paid once at ingest.
Why B: Situate at ingest; both retrieval channels benefit; the bill is one-time.
A [VG] nothing re-chunks at query time
C [VG] the cost lands at ingest, not per query
D [VG] the numbers are published and large

Q26 **B** — obj 3.5

The tell: 38k tokens, stable, always-used — under the ~200k skip-RAG threshold.
Why B: The pre-question of every RAG stem: is the corpus big enough to need RAG at all?
A [OE] training-scale solution; and rate cards change quarterly
C [OE] retrieval machinery for a corpus that fits in the prompt
D [CM] half-machinery for a corpus needing none

Q27 **A** — obj 3.5

The tell: Updated weekly → the index must follow, and staleness must be visible.
Why A: Indexes are caches of the corpus; caches need an invalidation story.
B [SF] the model will fluently cite the repealed procedure
C [OE] per-edit pipelines for weekly-cadence docs
D [TR] more stale chunks are not fresher chunks

Q28 **A** — obj 3.5

The tell: Overlap buys seam insurance and bills duplicate tokens — both sides named.
Why A: 3.5 grades knowing what each ingest dial buys AND bills.
B [VG] orthogonal techniques
C [VG] nothing indexed twice is free
D [VG] different failure mode entirely

Q29 **A** — obj 3.5

The tell: 97% floor versus 94.3 vs 98.1 — one option clears, one cannot.
Why A: Bind the constraint, then check the other currency has room. Both numbers, one sentence.
B [TR] guards slack latency, breaches the binding accuracy floor
C [SF] more of the wrong shortlist is not a corrected shortlist
D [CM] nothing needs removing; 3.0 s < 4 s

Q30 **C** — obj 3.6

The tell: Rows with keys + an exact-entity query — the matrix's most crimson cell.
Why C: Law 3: the data's shape picks. IDs → lookup, always.
A **[VG]** recall tuning cannot make similarity exact
B **[OE]** the fashionable hammer
D **[CM]** freshness doesn't fix approximateness

Q31 **C** — obj 3.6

The tell: Retrieval fetches text ABOUT lateness; only aggregation counts it.
Why C: RAG cannot count. Aggregate-language routing is the guardrail.
A **[SF]** fluent wrong numbers are the worst failure class
B **[VG]** not an embedding property
D **[VG]** no k counts rows

Q32 **A** — obj 3.6

The tell: Exact strings are literal-match territory.
Why A: Query shape tunes the fusion: codes → keyword-leaning; symptoms → semantic-leaning.
B **[CM]** deletes the strongest signal in the query
C **[TR]** leans away from the failing queries' needs
D **[CM]** depth is not channel balance

Q33 **A** — obj 3.6

The tell: Symptom prose vs procedure vocabulary — zero shared words, shared meaning.
Why A: Semantic search exists precisely for the synonym gap. Its mirror-image failure is Q on codes.
B **[CM]** SOP 214 answers it, under other words
C **[SD]** vandalising the corpus to meet one query
D **[VG]** length is not the mechanism

Q34 **D** — obj 3.6

The tell: Free text with rich metadata — write the metadata INTO the chunks, then both channels see it.
Why D: Situating context is not just for SOPs; it is how messy corpora become retrievable.
A **[SF]** summaries drop the disputed sentence you'll need verbatim
B **[CM]** raw chunks strand ids and dates outside both indexes
C **[CM]** informal text is exactly where synonyms rule

Q35 **B** — obj 3.6

The tell: Known categories, cheap decision, 40,000/day — D1's routing signature, not an agent's.
Why B: Domains compose: D1 picks the pattern, D3 picks the retrieval machine behind it.
A **[SD]** 'compare' without a metric or set
C **[OE]** loop latency and cost to re-solve a solved classification
D **[CM]** reinstates the fluent-wrong-number failure

Q36 **A** — obj 3.7

The tell: Many clients + changing list — the standard connector's exact home ground.
Why A: Count clients and churn. Both high → MCP.
B **[VG]** nothing here negotiates; these are function calls
C **[OE]** four welds, re-welded monthly
D **[CM]** vendored copies re-create the propagation problem

Q37 **A** — obj 3.7

The tell: Read data into context, no action — the resource's definition.
Why A: Tools act, resources inform, prompts template. Three primitives, one sentence.
B **[CM]** works, but spends the action primitive on a data need
C **[VG]** prompts are interaction templates, not document transport
D **[VG]** elicitation asks the USER for input

Q38 **D** — obj 3.7

The tell: Spec 2026-07-28: the DRIFT box, verbatim.
Why D: DRIFT discipline: state both readings, date the current one.
A **[VG]** they existed; they are deprecated, not fictional
B **[VG]** half-updated
C **[VG]** both moved

Q39 **A** — obj 3.7

The tell: One client, one endpoint, no churn — the weld is less machinery.
Why A: MCP is a multiplier on pairings; with one pairing there is nothing to multiply.
B **[OE]** standard for standard's sake
C **[CM]** the data is needed; the mechanism is the question
D **[VG]** there is no agent on the far side

Q40 **A** — obj 3.7

The tell: The far side has judgement — you exchange goals and proposals, not function calls.
Why A: A2A = brief + budget + verification. D1's handoff discipline crossing the company boundary.
B **[SD]** brittle and unauthorized
C **[CM]** a negotiation crammed into a lookup's shape loses the back-and-forth
D **[SD]** replaces an interface with an org-chart fantasy

Q41 **B** — obj 3.8

The tell: Any request uses at most 3 — the catalogue's exact use case.
Why B: Send the map first. Fetch on choice.
A **[CM]** caching discounts the bill; attention and window costs remain
C **[OE]** 7.8k tokens of standing insurance
D **[CM]** the other 27 are used, just not together

Q42 **C** — obj 3.8

The tell: Small, stable, always-used — the three words that flip the default.
Why C: The exception is on the Memory Map because the exam tests the flip, not the fashion.
A **[SF]** summaries silently drop the clause the dispute needs
B **[VG]** incoherent hybrid
D **[TR]** pays a round-trip to avoid a bill caching already cut ~90%

Q43 **D** — obj 3.8

The tell: Old tool results are the named clearable class; the API now trims them server-side.
Why D: clear_tool_uses_20250919 + memory tool: the long-run pattern, by name.
A **[CM]** workable but ignores the platform mechanism the stem's constraint points at (and DRIFT covered)
B **[SF]** restarts amputate the investigation's memory
C **[VG]** output budget; unrelated to input bloat

Q44 **A** — obj 3.8

The tell: Clearing invalidates cached prefixes — a priced trade, not a free lunch.
Why A: Every 3.8 mechanism has a bill; naming it is what 'evaluate' means.
B **[VG]** the cache interaction is the whole nuance
C **[VG]** the client keeps history; thinking-block policy is separate and configurable
D **[VG]** one re-write, not a permanent tax

Q45 **C** — obj 3.8

The tell: 1.4M tokens ends Plan P before taste begins; the latency check then clears Plan Q.
Why C: First check feasibility against the window, then price the survivor. Two numbers, done.
A **[CM]** the cases need the SOPs; that is the task
B **[CM]** physically impossible before it is stylistically debatable
D **[SF]** corpus-across-turns is monolith cosplay with worse recall

THE MEMORY MAP

Twenty-six things worth carrying into the room

THE LAWS & THE CONNECTOR

1. Law 1: every tool is a door — token cost (schema × every call), confusion cost (lookalikes), security surface. Trim by **logs AND consequence**.
2. Law 2: chunk along meaning — cut on headings, write the origin INTO the chunk, size ≈ query granularity.
3. Law 3: retrieval follows the data's shape — **ID → lookup · count → aggregate · procedure prose → hybrid · codes → keyword-lean · symptoms → semantic-lean**. RAG cannot count.
4. Law 4: you cannot fix what you cannot see — logs=WHAT, traces=WHERE, metrics=NORMAL?; SLAs are promises about **p95s**, not means.
5. Law 5: the map first, territory on demand — EXCEPT small + stable + always-used + cached (≲ 200k tokens) → monolith + caching wins.
6. MCP primitives: **tools act · resources inform · prompts template**; naming mcp__server__tool; discovery via */list.
7. 3.7 sentence: count **clients and churn** — both high → MCP; one client no churn → direct call; judgement on the far side → agent-to-agent (brief + budget + verify).

SPEC & API FACTS (DATED SEP 2026)

8. MCP 2026-07-28: **stateless**, _meta on every request, server/discover; **sampling & logging deprecated**; notifications opt-in (subscriptions/listen); transports **stdio + Streamable HTTP** (SSE gone); OAuth recommended remote.
9. Context editing (server-side): clear_tool_uses_20250919 — trigger ~100k, keep 3, exclude_tools, clear_at_least; clear_thinking_20251015; clearing can invalidate cache; memory tool saves essentials first.
10. Caching minimums by tier 512–4,096 (newest tier 512 · Sonnet 5 = 1,024 · Haiku 4.5 = 4,096); cache reads ~90% off.
11. Subagent caps carry over: depth 3 · concurrency 20 · max_budget_usd.

RAG — THE NUMBERS THAT DECIDE

12. Contextual retrieval: situating context before **embedding AND BM25**; failure-rate cuts **35% → 49% → 67%** (5.7 → 3.7 → 2.9 → 1.9%).
13. Rerank shape: retrieve ~150 → rerank → top-20; +~200 ms; top-20 beat 5 and 10.
14. Skip-RAG threshold: corpus ≲ 200k tokens (~500 pages) → whole corpus + prompt caching.
15. Weekly-updated corpus → scheduled re-index + version labels in chunks; stale citation must be VISIBLE.
16. Miss diagnosis by stage: wrong section = chunking · right-doc-wrong-rank = fusion/rerank · right-chunk-wrong-answer = prompt assembly. Fix the stage that dropped it.
17. Hybrid: keyword misses synonyms, semantic misses exact codes — each fails alone, silently, by query shape.

AUTH — THE GATE PASS

18. authN = who you are; **authZ = which bays**. The heist ghost: a VALID credential with a wildcard manifest.
19. Scope check in code, BEFORE the tool body; per-caller identities; least privilege; wildcards are findings.
20. Log every decision INCLUDING denials; dashboard denials-by-caller — a 03:00 spike with healthy latency is a probe.
21. stdio inherits the OS user; Streamable HTTP wants OAuth + scopes. “Never do X” = omit the tool, not a louder prompt.

OBSERVABILITY & THE DIALS

22. Stack: alerts (few, customer-felt) → dashboards → metrics (LOW cardinality — ids NEVER label metrics) → traces (one id per request) → structured logs (JSON, ids welcome).
23. The incident walk: alert → dashboard → metric → trace → log line. Mean 310/p95 2,900 = the mean lied.
24. 3.3 justification shape: constraint + dial + direction + number (“refunds err at ₹1,800 vs ₹4 → big model there; blended ≤ ₹6 holds”).
25. Timeouts: a hung dependency must cost a known maximum — timeout + one retry + honest degradation.
26. VG is D3's dominant family — when a word is fog, the glossary page beats re-reading the chapter.

indigo tokens · **gold** numbers · **crimson** traps · **green** correct moves · underline = the tell — read twice on Sun 1 Nov, alongside the other six maps.

THE MISS LEDGER

One row per miss — ten minutes, every session

Families: **VG** vocabulary gap · **OE** over-engineered · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped (+ **OP** over-permissive where auth stems earn it). Write the tell you skipped in your own words.

Date	Source	Q#	Obj	Family	The tell (one line)	Fixed?

Pattern rule: three **VG** rows → one more glossary pass beats any re-read. Three **CM** rows → slow down and underline. A wrong cell on the A3 matrix → redraw A3 from memory before the next sitting.